esprit

Se former autrement

2025/2026

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY : **Software Engineering**

MES-X Dependency Management

Prepared by:

Ala Eddine Mdalla

Supervised by:

Academic Supervisor: Wafa Boumaiza

Professional Supervisor: Khemiri Karima

FORVIA faurecia

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I validate the submission of the student's report:

Nom & Prénom /Name & Surname : Alaeddine Mdalla

Encadrant Entreprise/ Business site Supervisor

Nom & Prénom /Name & Surname : Khemiri Karima

Cachet & Signature / Stamp & Signature

FAURECIA INFORMATIQUE TUNISIE
Service RH
CUN - Imm. " Cercle de Bureaux "
7ème etage Bur. N°B7-1082 TUNIS-TUNISIE
Tel : 31 397 760

Encadrant Académique/Academic Supervisor

Nom & Prénom /Name & Surname : Boumaiza Wafa

Signature / Signature

Ce formulaire doit être rempli, signé et scanné/This form must be completed, signed and scanned.

Ce formulaire doit être introduit après la page de garde/ This form must be inserted after the cover page.

Acknowledgments

This work would not have been possible without the support, guidance, and kindness of the many people who accompanied me throughout this journey.

First, I would like to express my sincere gratitude to **Ms. Khemiri Karima**, **Mr. Abassi Chifa**, and **Ms. Gabsi Oumeima** for their continuous support, guidance, and encouragement during my internship at Forvia. Their professionalism, trust, and kindness made this experience both enriching and memorable.

I am also grateful to **Ms. Boumaiza Wafa** for her attentive supervision and the valuable advice she provided throughout this work. Her patience and encouragement were a constant source of motivation and played an important role in shaping this project.

I would like to thank the **MES-X team** for their collaboration and the welcoming environment they created. I am especially thankful to **GHAZOUANI Mohamed Neji**, **BEJAOUI Nawres**, **SALAH Houssam**, and **YOUAZ Ichrak** for their support, availability, and team spirit.

On a more personal note, I am deeply grateful to my family, who have always been my greatest support.

My first thought goes to my **late father, Sammi Mdalla**, who passed away last March. Although he is no longer here to see this work, his values, sacrifices, and belief in me remain the foundation of everything I have achieved. I carry his memory with me every day.

To my **mother, Hedia Ben Jouda**, thank you for your love, patience, and strength. You have always been my source of courage. To my **brother, Dhia Mdalla**, and my **sister, Wafa Mdalla**, thank you for your support and for always believing in me.

This accomplishment belongs to all of you.

Abstract

Large software teams working with Azure DevOps need a reliable way to trace the links between work items, pull requests, commits, branches, and release tags. Within the MES X.0 environment at Forvia, this traceability was previously built through a manual process that was slow, repetitive, and error-prone.

The **MES-X Dependency Management** project addresses this limitation by providing a read-only traceability service composed of a NestJS backend and a Vue 3 frontend. The backend integrates with Azure DevOps, traverses dependency graphs, resolves pull requests and commits, streams batch results, and identifies the tag context of each change. The frontend gives users an operational interface for launching analyses, following progress, and reviewing consolidated results.

This report presents the project context, the formal requirements, the proposed architecture, the implementation choices made for both backend and frontend, and the main technical outcomes of the internship.

Keywords: NestJS, Azure DevOps, Vue 3, dependency management, traceability, internship report.

Contents

List of Figures

List of Tables

List of Acronyms

Abbreviation	Definition
API	Application Programming Interface enabling communication between systems
REST	Architectural style for designing networked applications using HTTP
DTO	Data Transfer Object used to transfer structured data between application layers
E2E	End-to-End testing covering full system workflow
CI/CD	Continuous Integration and Continuous Delivery pipeline for automated software deployment
Git	Distributed version control system used for source code management
PR	Pull Request used for code review and integration in Git workflows
Epic	High-level work item in Azure DevOps grouping large functional features
PAT	Personal Access Token used for authentication in Git and API access
MES	Manufacturing Execution System for managing production operations
MES-X	Forvia MES platform context for the dependency management system
NFR	Non-Functional Requirement defining system quality attributes

Abbreviation	Definition
FR	Functional Requirement defining system behaviors and features
UC	Use Case describing system interactions with actors
NestJS	Node.js framework for building scalable backend applications
TypeScript	Strongly typed superset of JavaScript used in frontend and backend development
Vue	Progressive JavaScript framework for building user interfaces
Quasar	Vue-based UI framework for building responsive applications
Vite	Fast frontend build tool and development server
SSE	Server-Sent Events enabling real-time server-to-client communication
UML	Unified Modeling Language used for system design and visualization

General Introduction

Software systems today are no longer simple applications built by a single team. They are complex ecosystems composed of multiple services, tools, and workflows. In the MES X.0 environment at Forvia Informatique Tunisie, development is based on a microservices architecture supporting a large-scale Manufacturing Execution System deployed across several industrial sites. Azure DevOps plays a central role by providing unified support for planning, source control, code review, and CI/CD pipelines.

However, one of the main challenges in such an environment is traceability. A single feature can involve multiple work items, several pull requests, and a large number of commits distributed across different repositories. Understanding how these elements are connected is essential for ensuring reliability and maintainability.

Before this internship, traceability was handled mainly through manual navigation in Azure DevOps and supplementary spreadsheet tracking. This approach was time-consuming, error-prone, and difficult to maintain consistently across teams and releases.

The objective of this internship was to design and implement an automated traceability system capable of reconstructing dependency chains directly from Azure DevOps data. The goal was not only to reduce manual effort, but also to provide developers and release engineers with a clearer, more reliable view of relationships between requirements, code changes, and release versions.

Report Plan

This report is structured into five main chapters followed by a general conclusion.

The first chapter introduces the host company, Forvia Informatique Tunisie, and describes the industrial context of the MES X.0 project.

The second chapter presents an analysis of existing traceability approaches and highlights their limitations in complex industrial environments.

The third chapter defines the functional and technical requirements of the system.

The fourth chapter details the proposed solution, including its architecture and key design decisions.

The fifth chapter focuses on implementation, presenting how the system was developed and the results obtained.

Finally, the conclusion summarizes the outcomes of the internship and proposes future improvements and perspectives.

Chapter 1

Project Overview

1.1 Introduction

This chapter establishes the professional and technical context of the end-of-study internship project. It presents the host company, the industrial environment in which the work was carried out, the problem that motivated the project, and the main objectives assigned during the internship.

This context is important because the technical decisions described in the next chapters were shaped by concrete industrial constraints: a large microservices landscape, a growing number of Azure DevOps artifacts, and the operational need for faster and more reliable traceability.

1.2 Host Company

1.2.1 The Forvia Group

Forvia is one of the world's leading automotive technology groups, formed through the strategic merger of two major European industrial players: **Faurecia**, a global specialist in automotive seating, interiors, and emissions control technologies, and **HELLA**, a globally recognized leader in automotive lighting systems and electronic components. The merger created a group of exceptional scale, combining complementary technological portfolios and international footprints to serve automotive manufacturers across every major global market.

Forvia operates across more than 40 countries, employs over 150,000 people worldwide, and maintains a strong research and development focus aimed at accelerating

the transition toward sustainable, intelligent, and connected vehicles. Its product and technology portfolio spans seating systems, cockpit electronics, clean mobility solutions, lighting, and sensors — placing the group at the intersection of several of the most critical transformation vectors in the modern automotive industry.

1.2.2 Forvia Informatique Tunisie

The internship took place at **Forvia Informatique Tunisie**, the Tunisian IT and digital engineering hub of the Forvia Group. Established as a center of technical excellence within the group's global digital organization, this entity is responsible for the design, development, and long-term maintenance of the software systems that support industrial operations at Forvia's manufacturing facilities worldwide.

Forvia Informatique Tunisie concentrates a multidisciplinary team of software engineers, architects, DevOps specialists, and technical leads who work in close collaboration with operational and product teams across multiple Forvia business units. The entity contributes to a broad range of digital initiatives, including manufacturing execution systems, production monitoring platforms, enterprise integration solutions, and internal tooling designed to improve the efficiency and traceability of the group's software delivery processes.

1.2.3 The MES X.0 Division

The project was carried out in the MES X.0 division of Forvia Informatique Tunisie. This team develops and continuously improves a modern Manufacturing Execution System used in factories to manage, control, and track production in real time.

The MES X.0 platform is based on a microservices architecture composed of multiple services, each responsible for a specific role such as production planning, quality control, equipment monitoring, or operator assistance on the shop floor. These services communicate through APIs.

This design increases system flexibility. Each service can be developed and updated independently without affecting the entire platform. It also allows each factory to use only the modules it needs.

Today, the MES X.0 platform is deployed in several Forvia factories worldwide and processes real-time data from machines and operators.

However, this architecture also introduces significant development and maintenance complexity, which motivated the work presented in this report.

1.3 Project Context and Problem Statement

1.3.1 Development Environment at Forvia

At Forvia, **Azure DevOps** serves as the central platform for the software development lifecycle across all digital teams. It provides an integrated suite of tools covering the full range of development activities: sprint and release planning through work item boards, source code management through Git repositories, code review and integration through pull requests, automated build and deployment pipelines through Azure Pipelines, and artifact management for binaries and package versions.

Within the MES X.0 division specifically, Azure DevOps manages the development activities of all microservices simultaneously. A given sprint may involve planned work across ten, twenty, or more independent services, each progressing at its own pace with its own team, its own repository, and its own release cadence. The platform accumulates an ever-growing set of interconnected artifacts: work items describing requirements and tasks, pull requests integrating code changes, commits recording individual code modifications, branches isolating development streams, and repository tags marking release points.

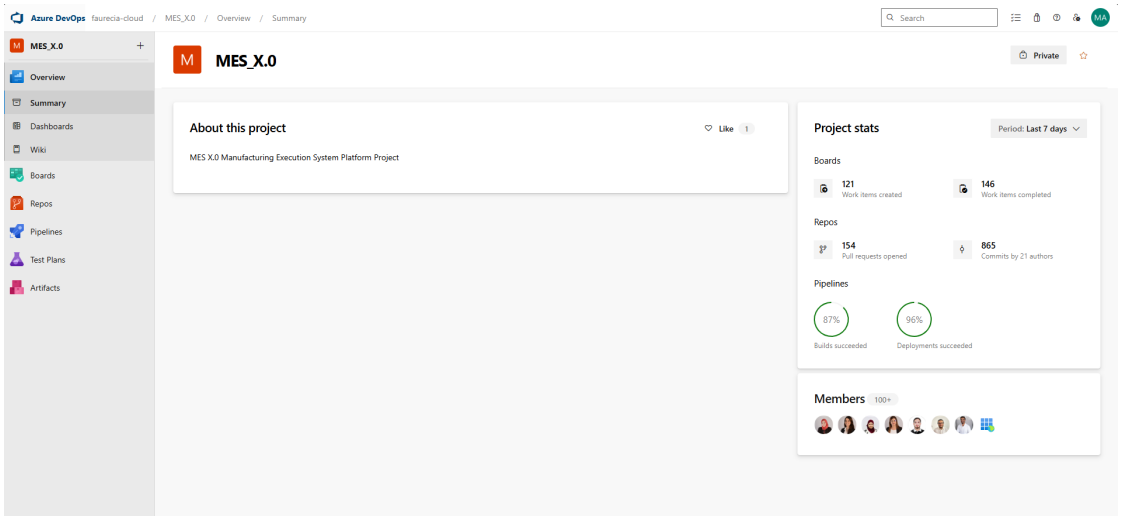

Figure 1.1: Azure DevOps environment used within the MES X.0 division, showing work item tracking and development workflow management

1.3.2 Deployment Environments and Versioning Strategy

In the MES X.0 ecosystem, each microservice is deployed across several environments that represent the standard software delivery lifecycle. These environments include **Development (DEV)**, **Quality Assurance (QUA)**, **Pre-production (PREPROD)**, and **Production (PROD)**.

Each environment plays a specific role in the validation and release process. The DEV environment is used for implementing and integrating new features. The QUA and PREPROD environments are used to validate functionality under conditions that are close to production, while the PROD environment hosts the live system used by manufacturing sites.

A key characteristic of the platform is that each microservice follows its own versioning across these environments. As a result, a service may not necessarily run the same version in DEV, QUA, PREPROD, and PROD at the same time, depending on its deployment and release cycle.

This versioning information is centralized in the `version.json` file located in the **infra repository**. This file defines the mapping between microservices and their deployed versions in each environment.

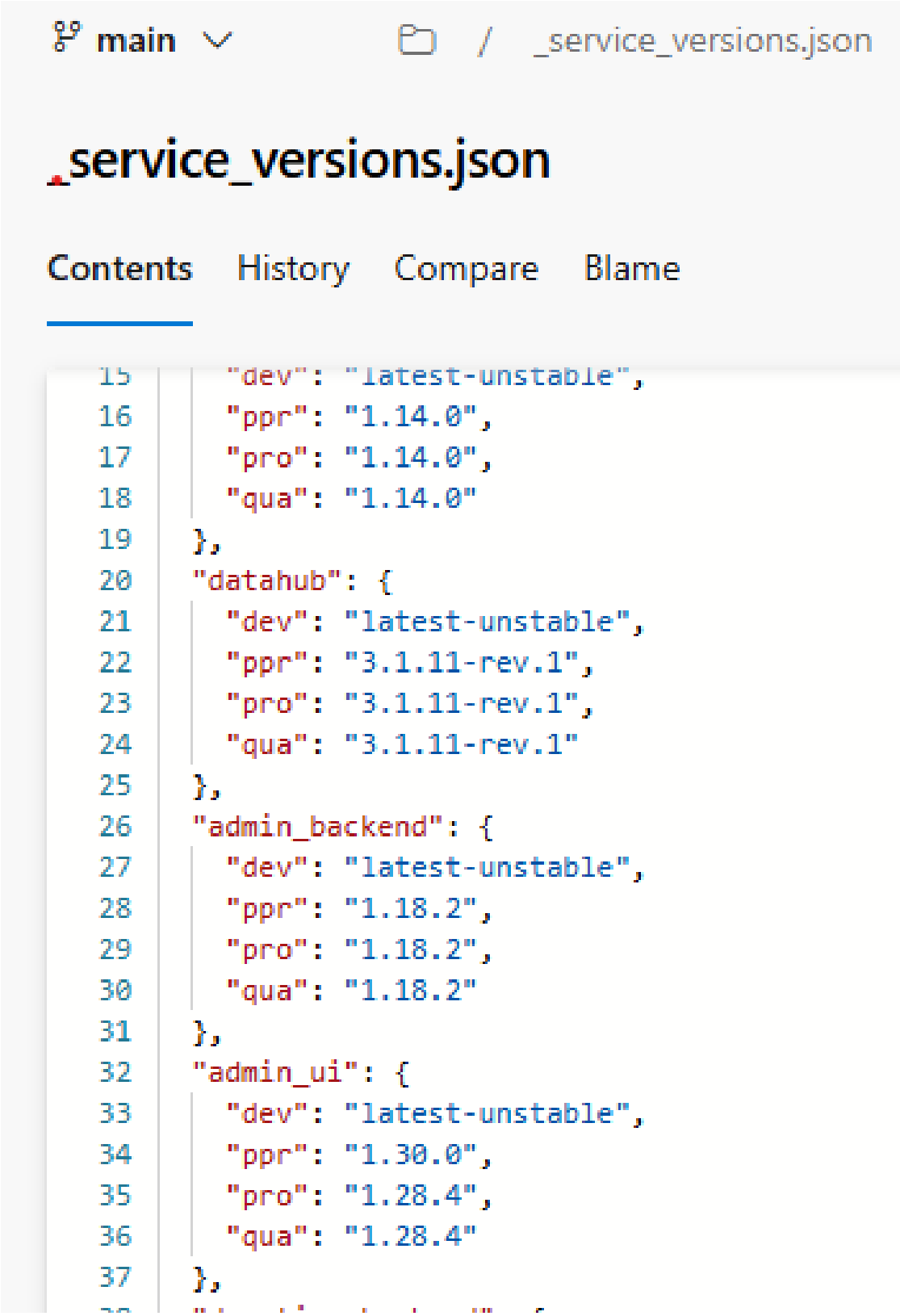

Figure 1.2: Excerpt from the version.json file showing microservice versions across DEV, QUA, PREPROD, and PROD environments

This file is essential for understanding the real state of the system at any given time. It is especially important during release analysis and dependency tracking, as it

helps identify which version of each service was active in each environment.

However, this environment-based versioning model also introduces additional complexity when analyzing system changes across services, particularly when trying to trace dependencies between work items and deployed versions.

1.3.3 The Traceability Challenge

In this multi-service development environment, a single user requirement, usually defined as an Epic or Feature in Azure DevOps, is not implemented in one place. It is broken down into smaller tasks that are distributed across different teams and services. Each task produces its own development artifacts such as branches, commits, pull requests, and finally a release tag when the change is delivered.

Because of this structure, understanding the full impact of a requirement, or identifying which requirements are affected by a change in a specific service, requires following the complete chain of artifacts.

For a release engineer preparing a production deployment, this often means answering questions such as:

- Which work items are included in this release, and are they completed?
- For each work item, which pull requests were used and in which repositories?
- For each repository, which commits were made, and do they belong before or after the release tag?
- Are there commits that are part of one release but not yet included in another environment?

Before this project, answering these questions was a manual process. A release engineer would start from a work item in Azure DevOps, follow its linked pull requests, open each one to identify the repository, then check the commit history to find the related changes. This had to be repeated for every work item, and the information was usually stored in spreadsheets for tracking.

This process had several limitations:

- **Time-consuming**: tracing a single feature across multiple services could take a full working day.
- **Error-prone**: manually linking information across Azure DevOps pages increased the risk of missing or incorrect data.

- **Inconsistent**: different engineers could follow different paths and get different results.
- **Not scalable**: as the number of microservices increased, the process became too slow for regular release cycles.

1.3.4 Project Motivation

These limitations created a clear and urgent need for an automated solution. The **MES-X Dependency Management** project was initiated within the MES X.0 division to address this need directly: to design and implement a system that would automate the reconstruction of dependency chains across Azure DevOps artifacts, transforming a time-consuming and error-prone manual process into a fast, reliable, and reproducible analytical capability.

The project was scoped as an end-of-study internship project, representing an opportunity to deliver genuine industrial value while applying and extending the software engineering knowledge acquired during the academic program.

1.4 Project Description

1.4.1 Project Objectives

The primary goal of the MES-X Dependency Management project is to design and implement an **automated dependency traceability system** that replaces the existing manual process with a reliable, efficient, and scalable solution integrated into the Azure DevOps ecosystem.

The project pursues the following specific objectives:

Automated Microservice Tag Identification

The system must automatically identify and associate the relevant microservice repository tags with work items by analyzing commit history and repository metadata. This capability eliminates the need for manual navigation through repository tag lists and commit histories, and ensures that the release context of each code change is determined programmatically and consistently.

Comprehensive Dependency Mapping

The system should be able to provide clear traceability links across the whole chain of development artifacts, starting from high-level work items and going through pull requests and commits, down to branches and repository tags. This mapping should cover all levels of the work item hierarchy and all types of artifacts, and it should remain consistent each time the analysis is run.

Significant Reduction of Manual Effort

By automating data retrieval, traversal, normalization, and aggregation, the system must reduce the time required to perform a complete dependency analysis from hours to seconds. This reduction must be sustained even for complex analysis scenarios involving multiple work items and many repositories.

Improved Data Accuracy and Consistency

The system should remove errors caused by manual data extraction and linking. All dependency data should come directly from a single reliable source, which is the Azure DevOps APIs, using a standard and repeatable process. This ensures that the results are accurate and remain the same every time the analysis is performed with the same input.

End-to-End Traceability

The system should allow a user to trace any work item from its initial description down to the exact commits and microservice tags associated with its implementation, all within a single analysis session. The user should not need to switch between tools or manually connect information to get a complete view of the dependencies.

Reusable and Scalable Architecture

The system should be designed with a modular and maintainable architecture. It should be easy to extend in the future to support new types of artifacts, new analysis features, or integration with other Forvia platforms, without having to redesign the whole system.

1.4.2 Technical Scope

The project includes the design and development of three main parts: a backend service, a frontend application, and an integration layer with Azure DevOps.

Backend Development

The backend was developed using **NestJS** with TypeScript. This framework was chosen because it supports modular design and helps keep the code clean and easy to maintain.

The backend is responsible for:

- Connecting securely to Azure DevOps using Personal Access Tokens.
- Transforming raw data from Azure DevOps into structured and typed data used inside the system.
- Traversing work item dependencies across different levels.
- Retrieving related pull requests, commits, branches, and tags.
- Supporting analysis starting from different entry points such as a single work item, multiple work items, or entire sprints.
- Providing a Sprint module that allows retrieving all work items related to one or more sprints, as well as their associated repository versions.
- Integrating an LLM-based module to review commits based on MES patterns and predefined rules, in order to verify that changes follow the expected development standards.
- Reducing the number of API calls using batch processing.
- Streaming results progressively using Server-Sent Events.
- Providing clear REST APIs with Swagger documentation for testing.

Frontend Application

The frontend was built using **Vue 3** with the Quasar framework. It provides a responsive interface for running and exploring dependency analysis.

The frontend is responsible for interacting with the backend and displaying results in a clear and structured way, including real-time updates during the analysis process.

Main features include:

- Visualization of work item dependencies in a clear and structured way.
- Support for different analysis entry points, such as a single work item, multiple work items, or entire sprints.
- Sprint-based views that allow users to explore all work items linked to one or more sprints, along with their related repository versions.
- Real-time progress updates using Server-Sent Events.
- Display of relationships between work items, pull requests, commits, branches, and tags.
- A Decision Dashboard that summarizes release and dependency results.
- A dedicated view for LLM-based commit review results, showing whether commits respect MES patterns and coding guidelines.
- A simple and practical design focused on daily use by engineers and release teams.

Azure DevOps Integration and Data Processing

The system uses Azure DevOps REST APIs as its only data source. This integration layer handles all communication with Azure DevOps and ensures that the data is retrieved correctly and efficiently.

It includes:

- Secure authentication using Personal Access Tokens.
- Batch processing and control of API requests to avoid rate limits.
- A dependency engine that explores relationships and removes duplicates.
- Error handling and retry mechanisms to avoid incomplete results.

1.4.3 Project Activities

The project followed the main steps below:

- **Requirements Analysis**: understanding the current manual process and defining system needs with engineers.

- **System Design**: defining the architecture, backend components, and data models.
- **Backend Development**: building the integration with Azure DevOps and implementing the dependency analysis logic.
- **Frontend Development**: creating the user interface and integrating real-time updates.
- **Testing**: verifying that the system works correctly and meets requirements.
- **Optimization**: improving performance and ensuring consistent results.
- **Deployment Preparation**: preparing Docker setup, configuration, and documentation.

1.5 Conclusion

This chapter presented the project context and explained why dependency traceability became a real issue. In a microservices environment, manual analysis is no longer efficient or reliable for release preparation.

The next chapter will define the system requirements and describe what the solution must achieve.

Chapter 2

Study of Existing Solutions

2.1 Existing Traceability Solutions in Azure DevOps

Before developing a custom solution, it is worth examining what traceability options already exist within the Azure DevOps ecosystem. Table 2.1 presents a comparison of three commonly used approaches.

Table 2.1: Comparison of existing traceability solutions in Azure DevOps

Solution	**Advantages**	**Limitations**
Azure DevOps Native Features	No additional setup required, basic visualization of relationships, Microsoft-maintained	Shallow dependency analysis only, no automated commit/tag extraction, limited programmatic access
Azure DevOps MCP Server	Structured API access, external tool integration, standardized query interface	Manual analysis logic required, no built-in tag extraction, limited scalability
Custom Scripting Solutions	Fully customizable, tailored data extraction	High development effort, lack of standardization, difficult to scale and reuse

While these existing solutions provide some level of traceability support, they fall short in automation depth and scalability for microservice architectures. None of them addresses the core problem of efficiently mapping dependencies across multiple repositories and services, which is the primary motivation for developing the system described in this report.

2.2 Current Dependency Tracking Process

This chapter analyzes the manual workflow used prior to the introduction of the MES-X Dependency Management system. Understanding this existing process is essential, as the limitations identified here directly influence the design of the proposed solution.

In the MES X.0 environment, the process typically begins with a **user story** created in Azure DevOps. Each user story is often linked to multiple child tasks. In turn, each task may generate its own pull requests, commits, and release references across different repositories.

Although Azure DevOps stores all these artifacts, it does not provide a unified or consolidated view of their dependency relationships. As a result, developers must manually reconstruct the dependency chain.

Manual Dependency Analysis Workflow

In practice, developers preparing a dependency report must follow a repetitive sequence of steps for each user story:

1. Open the work item in Azure DevOps and identify all linked child tasks and related items.
2. Navigate to the **Development** panel of each item to locate associated pull requests.
3. Open each pull request individually and access its **Commits** tab.
4. Analyze commit messages and modified files to determine the impacted microservice and extract version tags.
5. Manually record all relevant data (work item ID, pull request number, commit hash, microservice name, tag) in a spreadsheet.
6. Repeat the same process for every child work item linked to the main user story.

During a typical sprint, which may include multiple user stories and several pull requests per story, this process can consume a significant portion of a working day. Each dependency must be manually identified, verified, and recorded. As the number of services and work items increases, this manual approach becomes increasingly time-consuming and repetitive.

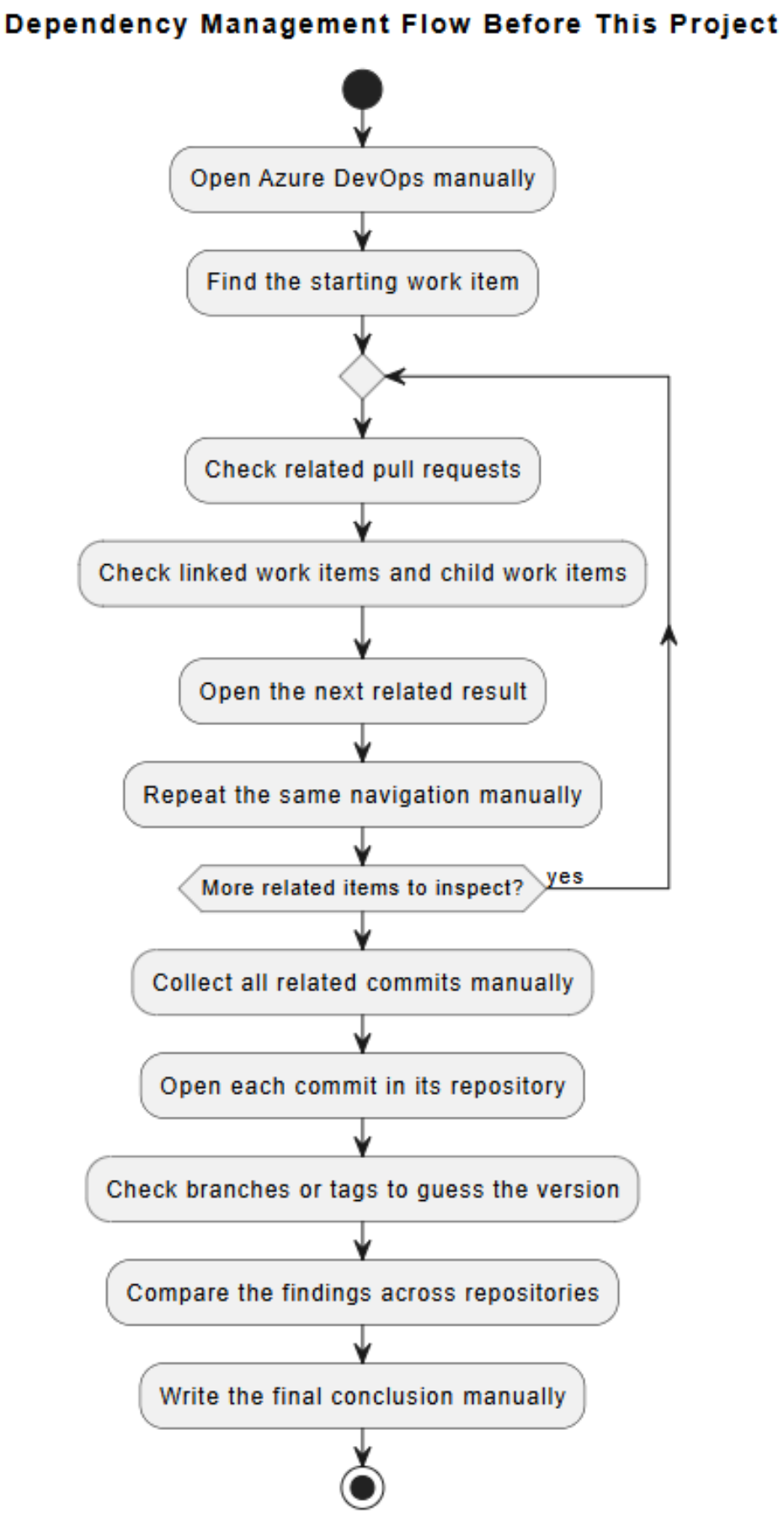

Figure 2.1: Manual dependency tracking workflow before the MES-X Dependency Management system

2.3 Illustrative Example

Consider a user story such as *"Update shared configuration service for a specific environment."* This change may involve multiple components, including the configuration service, a dependent API gateway, and deployment adjustments.

In practice, this single user story is divided into several child tasks, each associated with its own pull request and set of commits.

To construct a complete dependency view, the developer must:

- Inspect each pull request,
- Identify the affected repository,
- Review associated commits,
- Extract and compare release tags.

Additionally, commit tags must be analyzed to determine the most recent release version. It is also necessary to verify whether commits belong to a tagged release or fall between two versions.

Even in relatively simple scenarios, this requires navigating multiple pull requests and analyzing numerous commits. When repeated across an entire sprint, the workload becomes substantial and error-prone.

2.4 Limitations and Their Impact

The manual dependency tracking process introduces several key challenges:

Time Consumption. Generating a complete dependency report for a sprint can take several hours or even an entire day. Most of this time is spent navigating tools and manually collecting data rather than focusing on development tasks.

High Risk of Errors. The process relies entirely on manual verification. Missing a commit, overlooking a repository, or misidentifying a tag can result in incomplete or incorrect dependency reports, potentially affecting release decisions.

Limited Scalability. The MES X.0 platform consists of numerous microservices, with new ones being added continuously. As the system grows, the manual approach becomes increasingly difficult to maintain and scale.

Lack of a Single Source of Truth. Dependency data is typically stored in spreadsheets, leading to inconsistencies. Different engineers may produce different results for the same feature, with no centralized mechanism to validate or reconcile discrepancies.

Overall, these limitations negatively impact both efficiency and reliability. They clearly highlight the need for an automated solution, which will be introduced in the next chapter through the system requirements.

Chapter 3

Requirements Specification

3.1 Introduction

This chapter describes what the MES-X Dependency Management system must do and how well it must do it. The goal at this stage is to describe the expected behavior independently of implementation choices, so the architecture and code can later be measured against a clear baseline.

The requirements come from studying the manual workflow described in the previous chapter, talking with developers and release engineers, and watching traceability tasks being carried out in the MES X.0 environment. They cover both what the system must do and the quality bar it must meet in daily use.

3.2 Functional Requirements

The functional requirements define the main capabilities of the MES-X Dependency Management system. They describe how the system processes Azure DevOps data to build a complete traceability view across work items, source code, and release information.

- **FR-01 Pull Request Resolution**
  The system shall retrieve pull requests from Azure DevOps and associate them with their corresponding work items. This enables the system to establish a consistent link between work tracking elements and source control changes, which forms the entry point of the traceability pipeline.
- **FR-02 Commit and Tag Contextualization**
  The system shall extract commits linked to each pull request and add contextual

information, including branch details and repository tags. This contextual information is required to determine the position of each change within the release lifecycle and to support release-level analysis.

- **FR-03 Dependency Graph Construction**
  The system shall construct a dependency graph starting from one or multiple root work items. It shall traverse all related entities through Azure DevOps relationships, including parent-child links, pull request associations, and commit references, in order to reconstruct the full dependency chain.

- **FR-04 Aggregation and Decision Output**
  The system shall aggregate all retrieved entities into a unified data model. This includes normalization of work items, pull requests, and commits, as well as removal of duplicates caused by multi-path traversal. The final output shall provide a structured view that supports release analysis and decision-making.

- **FR-05 End-to-End Read-Only Traceability**
  The system shall provide a complete traceability view across all relevant artifact levels: work items, pull requests, commits, branches, and tags. All interactions with Azure DevOps shall be strictly read-only, ensuring that no modification, creation, or deletion of external data is performed by the system.

- **FR-06 Sprint-Based and Multi-Sprint Dependency Resolution**
  The system shall support dependency analysis starting from one or more sprints, as well as from work items belonging to multiple sprints. It shall retrieve all related work items and trace their dependencies across pull requests, commits, and repository versions.

  The system shall also identify the corresponding release versions associated with these work items in order to support validation of transitions between environments (e.g., development, pre-production, production). This enables the system to determine which changes are required to move a set of work items to the next environment.

3.3 Non-Functional Requirements

Non-functional requirements define the quality attributes that the system must respect to operate correctly in an industrial MES X.0 environment. They ensure that the system is not only functionally correct, but also performant, scalable, reliable, maintainable, and secure.

In this project, each non-functional requirement was directly translated into implementation decisions within the system architecture. Their validation was performed using a combination of unit tests, integration tests, runtime observation, and deployment-level verification.

3.3.1 Performance

Performance ensures efficient dependency analysis even in large-scale distributed data.

- **NFR-PE-01 Graph Traversal Efficiency**: The system uses graph traversal strategies in the backend to explore relationships between work items, pull requests, and commits while avoiding redundant exploration. This requirement is supported by the Work Item and Commit services, which combine traversal logic with in-memory deduplication and visited-state management.

  This requirement was verified through service-level tests and runtime observation of traversal behavior during dependency analysis.

- **NFR-PE-02 API Call Optimization**: Azure DevOps API requests are optimized through batching and controlled concurrency. In particular, the backend uses the `p-limit` library to cap concurrent external calls in traceability-critical services such as work item traversal, commit resolution, and tag resolution.

  This behavior was verified through code-level inspection of the concurrency control implementation, unit tests around the affected services, and runtime observation showing reduced request bursts and more stable execution.

- **NFR-PE-03 Caching and Deduplication**: Processed entities are cached and deduplicated during traversal to avoid recomputation and repeated API calls. This is implemented through local caches, aggregation normalization, and repository-level deduplication utilities in the backend.

  Its effectiveness was verified through repeated executions on identical inputs, which produced stable outputs with lower redundant processing.

- **NFR-PE-04 Progressive Feedback**: Long-running operations provide streaming updates to the frontend, improving perceived responsiveness. This is implemented through Server-Sent Events, with a dedicated streaming service that emits progressive batches using the `text/event-stream` protocol.

  This requirement was validated through end-to-end execution of batch analysis scenarios in which the frontend received progressive updates before full completion.

3.3.2 Scalability

Scalability ensures the system can handle increasing data volume and repository complexity.

- **NFR-SC-01 Algorithmic Scalability**: The combination of graph traversal, caching, and controlled concurrency ensures that execution cost grows with the actual dependency graph size rather than with redundant repeated exploration.

  This was verified through multi-root and sprint-based analysis scenarios involving multiple work items and repositories.

- **NFR-SC-02 Microservices Architecture**: The system is divided into domain-focused backend services and modules, including Work Item, Pull Request, Commit, Sprint, Decision, and LLM modules. This modular NestJS architecture supports isolated evolution, clearer responsibilities, and future scaling of the processing pipeline.

  This was verified during implementation by integrating new modules without redesigning the existing traceability flow.

- **NFR-SC-03 Containerized Deployment**: Docker is used to package both backend and frontend applications, ensuring reproducible builds and deployment behavior across environments.

  This was verified through the presence of Docker build scripts and Dockerfiles in both applications, as well as their integration into the delivery workflow.

- **NFR-SC-04 Dapr Integration**: Dapr is used as an integration component for secret retrieval and distributed application support. In the current solution, it is concretely used to retrieve Azure DevOps secrets from the configured secret store and it prepares the platform for wider distributed-service evolution.

  This was verified through the implemented backend integration based on `DaprClient` and the application configuration services.

3.3.3 Reliability

Reliability ensures consistent system behavior under all execution conditions.

- **NFR-RE-01 Deterministic Output**: The system is designed to produce identical results for identical inputs regardless of execution order. This is supported by deterministic normalization, deduplication rules, and stable aggregation logic in the backend.

This requirement was verified through repeated runs using the same inputs and by checking that the resulting traceability outputs remained consistent.

- **NFR-RE-02 Fault Isolation**: Failures in Azure DevOps API calls are isolated within the backend integration and service layers to avoid uncontrolled propagation across the whole application.

  This is supported by dedicated service boundaries, exception handling, and structured logging in the NestJS backend.

- **NFR-RE-03 Retry Mechanism**: Transient failures are handled using controlled recovery logic and guarded external-call handling while preserving partial processing whenever possible.

  This was verified through service tests and runtime observation of backend behavior during unstable or partial-response situations.

- **NFR-RE-04 Data Consistency**: All processed entities remain consistent across transformation and aggregation stages. This is supported by DTO-based contracts, typed mapping, and centralized aggregation logic.

  This requirement was verified through unit tests, API checks, and frontend-backend consistency validation during traceability workflows.

3.3.4 Maintainability

Maintainability ensures the system remains easy to evolve and extend.

- **NFR-MA-01 Modular Design**: Each module follows a modular design aligned with the Single Responsibility Principle, ensuring clear separation of concerns between traversal, commit processing, sprint handling, decision aggregation, and LLM review.

  This was verified during development by the existence of dedicated backend modules and services with isolated responsibilities.

- **NFR-MA-02 Clean Architecture**: Business logic, API integration, and data processing layers are separated to improve readability, testability, and long-term evolution. This separation is reflected in the NestJS service structure and in the dedicated configuration, integration, and utility layers.

  This requirement was verified through code review, module boundaries, and the ability to test controllers and services independently.

- **NFR-MA-03 Reusable Components**: Core services are designed to be reusable across multiple MES modules. Examples include shared streaming, logging, Azure DevOps integration, and aggregation utilities.

  This was verified by reuse of common services across several backend domains.

- **NFR-MA-04 Standardization**: Consistent naming conventions, DTO contracts, linting rules, and formatting rules are enforced across the codebase. This is supported by ESLint, Prettier, and typed DTO usage in both backend and frontend projects.

  This requirement was verified through lint scripts, formatting scripts, and the standardized project structure used during implementation.

3.3.5 Security and Data Integrity

Security ensures controlled and safe access to industrial data.

- **NFR-SE-01 Token-Based Authentication**: Azure DevOps access is secured using Personal Access Tokens whose secret names are provided through configuration and resolved at runtime from the secret store. The PAT is handled only by the backend and is never exposed to the frontend.

  This requirement was verified through the implemented Azure DevOps connection service, which retrieves the secret through Dapr before creating the authenticated Azure DevOps client.

- **NFR-SE-02 Frontend Route Protection**: The frontend includes a route-protection mechanism based on Vue Router metadata, a global `beforeEach` navigation guard, and an unauthorized fallback page. This mechanism allows views to be protected whenever `requiresAuth` is enabled.

  This was verified through the implemented authentication and authorization flow in the frontend router and authentication store.

- **NFR-SE-03 Secret Management**: Sensitive credentials are managed through secure configuration mechanisms using `ConfigService`, environment variables, and Dapr secret store integration.

  This was verified through the backend configuration modules and runtime secret resolution logic.

- **NFR-SE-04 Backend-Only External Access**: All communication with Azure DevOps is handled exclusively by the backend layer through dedicated integration

services. The frontend communicates only with backend endpoints and does not directly call Azure DevOps.

This requirement was verified through the architecture, the implemented router/API flow, and the backend integration services.

- **NFR-SE-05 Read-Only Policy**: The system is strictly read-only with respect to Azure DevOps, ensuring that no external artifacts are created, modified, or deleted during traceability analysis.

  This was verified by reviewing the exposed backend functionality, which focuses on retrieval, traversal, enrichment, and aggregation only.

3.3.6 Verification Scope

This chapter defines the non-functional targets and the technical mechanisms used to satisfy them. The concrete execution evidence (test outcomes, deployment traces, and pipeline-level proof) is consolidated in the Implementation chapter to avoid redundancy and keep this specification focused on requirements and verification criteria.

3.4 Use Case Model

The use case model shows how the actors interact with the system to carry out traceability and analysis tasks. It complements the requirement lists with a behavioral view of the system.

3.4.1 Actors

The use case diagram highlights the two primary actors shown in the figure: a Developer and an LLM Provider. The diagram is intended to make the main workflow and intelligence flow visible at a glance.

Developer and Release Engineer

The primary user. Developers use the system to understand the impact of a work item on the codebase. Release engineers use it to check release scope and confirm whether changes sit on the correct side of a release boundary.

This actor starts analysis sessions, chooses the analysis mode, reviews results, and uses the Decision Dashboard output to support release, integration, and escalation

decisions.

LLM Provider

An external AI service used by the system to review commits and generate intelligent insights. This actor supports the "Review Commit with LLM" use case and is shown in the diagram as a cross-cutting service.

Additional Actors

Other actors are part of the overall model but are not illustrated in this view. The Administrator manages operational configuration and availability, and Azure DevOps is the external data source for work items, pull requests, commits, branches, and tags.

3.4.2 Use Case Descriptions

The following use cases describe what the system does from each actor's perspective.

Work Item Use Cases

- **UC-01 Analyze Single Work Item**: The user provides one work item ID. The system traverses the full dependency graph, resolves related pull requests and commits, enriches commits with branch and tag context, and returns a complete normalized trace result.
- **UC-02 Analyze Multiple Work Items**: The user provides several work item IDs. The system traverses each root, merges and deduplicates the results, and returns one consolidated trace. This supports release scope analysis across a sprint or milestone.
- **UC-03 Stream Work Item Tasks**: The user starts a batch analysis session. The system streams resolved work item tasks progressively so progress is visible in real time, and items can be selected for further analysis before the full dataset has loaded.

Pull Request Use Cases

- **UC-04 Retrieve Pull Request Work Items**: Given a pull request identifier, the system returns the work items formally linked to that PR on the Git side, along with the normalized PR summary including the repository name.

Commit Use Cases

- **UC-05 Retrieve Commits by Work Item**: Given a work item ID, the system returns the deduplicated list of commits linked to that item, resolved through direct links and pull request expansion.
- **UC-06 Retrieve Commits by Pull Request**: Given a pull request ID, the system returns the list of commits merged as part of that PR.
- **UC-07 List Repository Branches and Tags**: Given a repository ID, the system returns the available branch and tag references needed to interpret commit positions.
- **UC-08 Resolve Commit Tag Context**: Given a repository and a commit, the system determines the most recent tag reachable from that commit on the relevant branch.
- **UC-09 Batch Resolve Commit Tag Context**: Given a repository and multiple commits, the system resolves the tag context of all specified commits in one operation to reduce API usage on large graphs.

Aggregation and Decision Use Cases

- **UC-10 Generate Dependency View**: After a trace is complete, the system aggregates all collected data — work items, pull requests, commits, and tag context — into a single decision-ready view. Work items are grouped by completion state and repositories are classified by their commit position relative to the target release.
- **UC-11 Configure System**: The Administrator updates environment-level configuration, including Azure DevOps connection settings, API credential management, and deployment parameters.

3.4.3 Use Case Diagram

Figure 3.1 shows the complete use case diagram, with the primary actors and all eleven use cases.

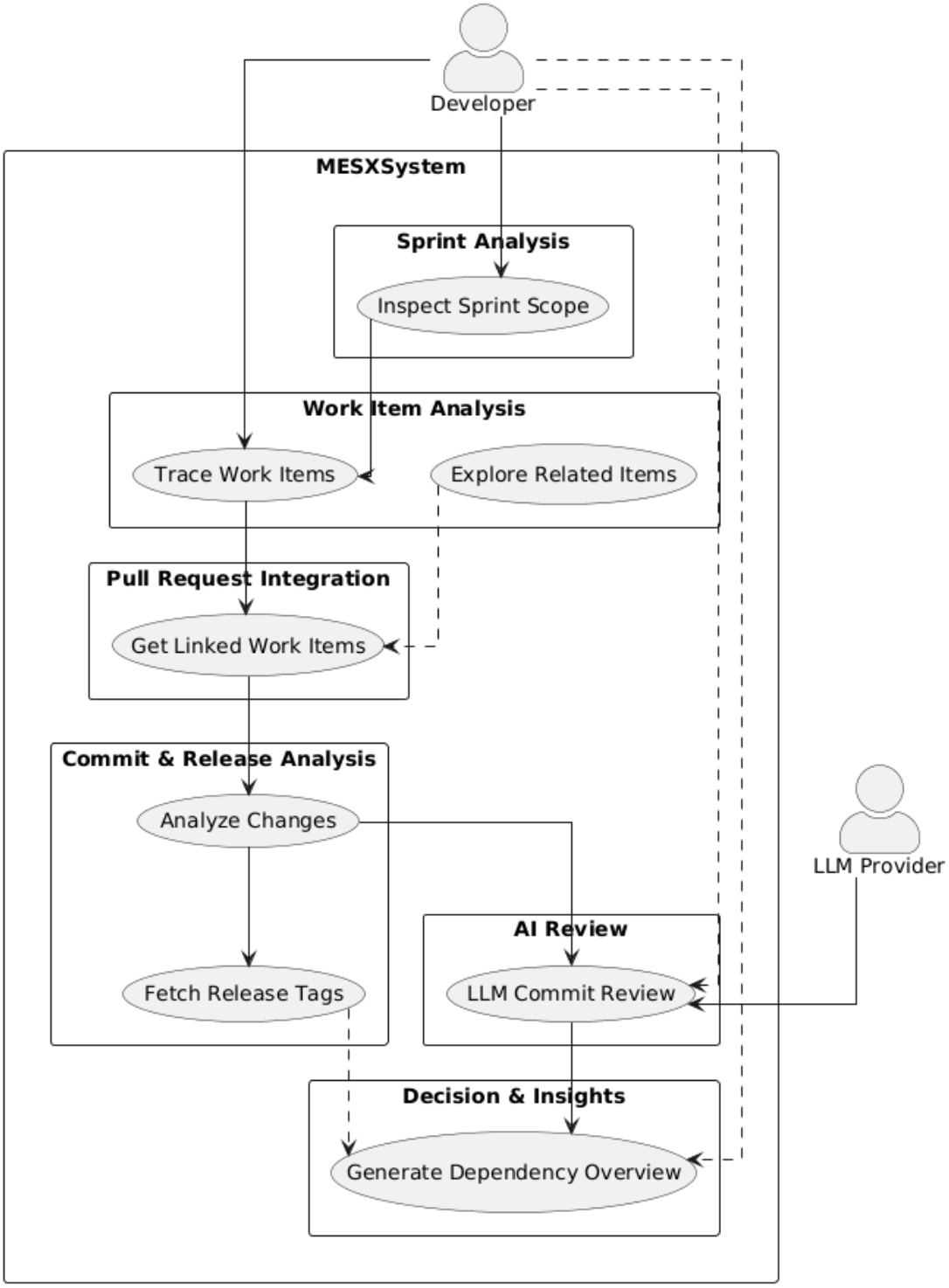

Figure 3.1: Use Case Diagram of the MES-X Dependency Management System.

3.4.4 Scope Boundaries

Two boundaries are worth stating clearly.

First, as set out in **FR-05**, all use cases are read-only. The system does not create,

modify, or delete anything in Azure DevOps. Azure DevOps appears in the model only as a data source.

Second, notification and alerting features, mentioned in some internal notes as possible future additions, are out of scope here. This specification covers only the traceability and analysis capabilities described above.

3.5 Requirements Traceability Summary

Table 3.1 shows how each functional requirement group maps to a system component. This makes clear which part of the system is responsible for which requirements, and creates the link between specification and design.

Table 3.1: Functional Requirements to System Component Traceability Matrix

Requirement Group	**Requirement IDs**	**Responsible Component**
Pull Request Resolution	FR-01	Pull Request Resolution Component
Commit and Tag Contextualization	FR-02	Commit Aggregation Component
Dependency Graph Construction	FR-03	Work Item Management Component
Aggregation and Decision Output	FR-04	Decision Aggregation Component
End-to-End Read-Only Traceability	FR-05	All components, Trace UI
Sprint-Based and Multi-Sprint Resolution	FR-06	Sprint + Work Item + Commit Components

3.6 Conclusion

This chapter set out the full requirements for the MES-X Dependency Management system. The functional requirements cover five areas: work item traceability, pull request resolution, commit enrichment, data aggregation, and analytical reporting.

The non-functional requirements set the bar for performance, scalability, reliability, maintainability, and security. The use case model translates those requirements into eleven concrete actor-system interactions.

Together, these requirements form the reference baseline for the rest of the report. They define not just what the system must do, but the level of performance, reliability, and maintainability expected of it in real use.

The next chapter presents the proposed architecture and explains how the chosen design addresses these requirements through a layered structure and a specialized traceability pipeline.

Chapter 4

Proposed Solution

4.1 Introduction

As discussed in the previous chapter, Azure DevOps holds all the data a team needs to trace a feature from planning through to deployment. The problem is not that the data is missing; it is that it is scattered. Work items, pull requests, commits, branches, tags, and sprint iterations each live in their own corner of the platform, with no built-in view that ties them together into a single coherent picture.

In practice, this means engineers and project managers must manually jump between multiple artifacts every time they want to understand the full lifecycle of a change. On a small project, that is manageable. In a microservices environment with dozens of repositories and hundreds of work items per sprint, it quickly becomes both time-consuming and unreliable.

This project addresses that gap directly. Instead of replacing Azure DevOps, we built a **centralized dependency traceability system** on top of it: a layer that collects data from Azure DevOps APIs, rebuilds relationships between artifacts, and exposes the result through one dedicated interface.

In practice, the system:

- fetches data from Azure DevOps APIs,
- rebuilds the dependency chain between artifacts,
- adds contextual metadata to raw data (branch position, release tags, repository status),
- supports sprint-driven work item selection as a natural entry point,

- offers optional AI-assisted commit review for deeper code-level insight,
- and displays results in a structured, actionable interface.

Three objectives guided the design:

- **Completeness.** The full dependency chain, from planning artifacts to code changes, should be visible in one place.
- **Consistency.** Data from different APIs and repositories must be standardized before processing, so results stay predictable even when teams configure projects differently.
- **Actionability.** The system should not only display data; it should help users answer concrete questions, such as whether a feature is ready for release or whether specific repositories still carry risk.

The rest of this chapter presents the architecture, the key design decisions, the core components, and the end-to-end flow that turns raw Azure DevOps data into structured traceability output.

4.2 Global System Architecture

The system is organized into three layers, each with a clear responsibility: the user interface, backend processing logic, and external integrations. Keeping these layers separate means that a change in one area, such as a breaking update in an Azure DevOps API, does not ripple through the rest of the system.

4.2.1 Presentation Layer: Trace UI

The Trace UI is the web interface through which users interact with the system. Its main role is to let users trigger dependency analysis, monitor progress in real time, and explore results in a structured way.

Beyond visualization, the Trace UI handles user input, calls the backend, and organizes returned data into analysis-friendly views. It supports four modes:

- single work item analysis,
- batch analysis of multiple items,

- sprint-driven selection, where the user picks a sprint and the system retrieves the associated work items automatically,
- and batch-related analysis, which combines dependency graphs across several items at once.

Each mode targets a different use case, from a quick spot-check on one feature to a full sprint-level analysis before a release.

4.2.2 Application Layer: Backend Services

The backend is where processing happens. It sits between the Trace UI and Azure DevOps, handling everything that requires business logic:

- constructing the dependency graph from work item relationships,
- normalizing data coming from different APIs into a consistent internal format,
- enriching artifacts with contextual information (branch position, tag data, repository status),
- retrieving work items associated with a sprint,
- coordinating optional LLM-assisted commit review,
- and computing the final release-readiness decisions.

Internally, the backend is organized into domain modules (Work Item, Pull Request, Commit, Sprint, Decision, and LLM), each responsible for one slice of the pipeline.

4.2.3 Integration Layer: Azure DevOps APIs

The integration layer handles all communication with Azure DevOps. It relies on two APIs:

- the **Work Item API**, used to retrieve work items, their relationships, and sprint iteration data,
- and the **Git API**, used to fetch pull requests, commits, branches, and tags.

When optional AI-assisted review is requested, the system also calls the Gemini API through a dedicated backend module.

All API calls go through shared utilities for authentication, request formatting, and error management. This keeps the rest of the system insulated from service-specific details.

4.2.4 Layer Interaction

Communication between layers follows a strict one-way flow:

$$\text{UI} \rightarrow \text{Backend} \rightarrow \text{Azure DevOps}$$

The Trace UI never calls Azure DevOps directly. All requests go through the backend first. This makes the system easier to test, easier to secure, and easier to maintain as APIs evolve.

4.2.5 Logical Architecture View

The logical architecture organizes the system into distinct functional modules, each encapsulating a specific responsibility within the traceability pipeline. Figure 4.1 illustrates the relationships between these modules and their interactions with external systems.

At the presentation tier, the **Traceability UI** serves as the sole entry point for end users. It initiates processing requests and presents structured results, but delegates all business logic to the backend services.

The backend services tier contains four core modules. The **Work Item Processing** module handles the retrieval and traversal of work item dependency graphs from Azure DevOps. It serves as the starting point for traceability analysis and orchestrates the flow of data to downstream modules. The **Pull Request & Commit Analysis** module resolves pull requests linked to work items, retrieves associated commits, and enriches them with repository metadata such as branch positions and release tags. The **Traceability Engine** aggregates data from previous stages and applies business rules to determine release readiness. It produces structured decision outputs that indicate which artifacts are complete, which repositories are deployment-ready, and where gaps exist. The **AI Analysis** module provides optional commit-level review capabilities by interfacing with external language models.

External systems include **Azure DevOps**, which provides all project artifacts through its REST APIs, and the **AI API (Gemini)**, which supports on-demand

code review when requested by users. The Work Item Processing module communicates with Azure DevOps to fetch planning artifacts, while the AI Analysis module sends review requests to the Gemini API. These external dependencies are isolated to specific modules to limit the impact of changes in third-party services.

Data flows sequentially through the processing chain. The UI triggers the Work Item Processing module, which in turn invokes the Pull Request & Commit Analysis module. Results are forwarded to the Traceability Engine for aggregation and decision generation. The AI Analysis module operates independently and only executes when explicitly requested by the user, ensuring that core traceability logic remains deterministic and independent of external AI services.

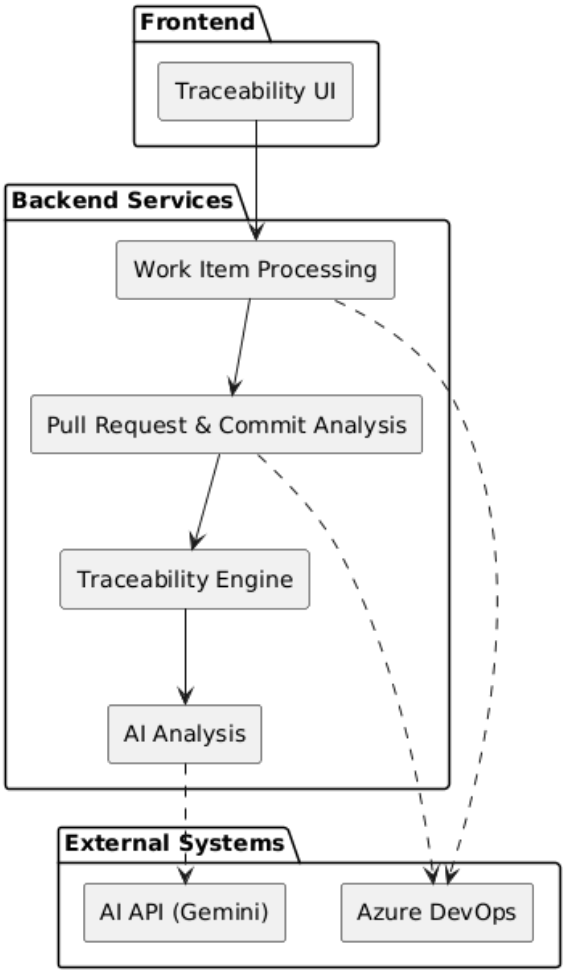

Figure 4.1: Logical architecture showing the main functional modules and their dependencies

4.2.6 Physical Deployment Architecture

The physical architecture defines how system components are deployed, hosted, and connected in the production environment. Figure 4.2 presents the deployment topology.

The system follows a cloud-native deployment model hosted on **Microsoft Azure**. Users access the system through standard web browsers on client machines, communicating with cloud-hosted services over HTTPS.

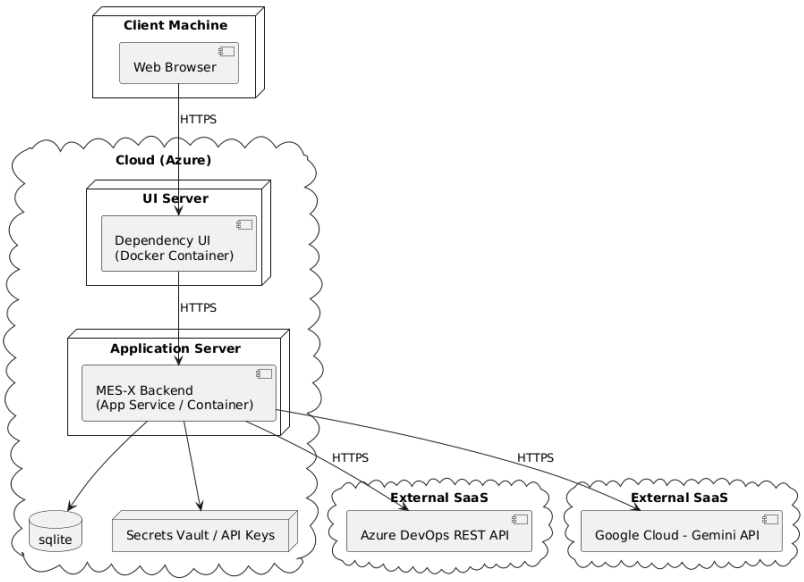

Figure 4.2: Physical deployment architecture showing hosting infrastructure and external integrations

Within the Azure cloud environment, the **UI Server** hosts the Trace UI as a containerized application deployed using Docker. This container serves the frontend application to client browsers and forwards API requests to the application server. The **Application Server** hosts the backend services, deployed either as an Azure App Service or as a containerized instance. This server contains all backend processing logic, including work item analysis, pull request resolution, commit aggregation, and decision generation.

The backend relies on an embedded **SQLite** database for local data persistence. SQLite was chosen for its simplicity and sufficient performance characteristics for this use case, eliminating the need for a separate database server. Sensitive credentials, including Azure DevOps personal access tokens and Gemini API keys, are stored securely in an **Azure Secrets Vault**, which the application server accesses at runtime.

External integrations are established with two Software-as-a-Service platforms. The **Azure DevOps REST API** provides access to work items, pull requests, commits, branches, and tags. The **Google Cloud - Gemini API** offers language model capabilities for optional commit-level code review. Both external services are accessed over HTTPS, and authentication is managed through API keys retrieved from the secrets vault.

All communication between components occurs over encrypted HTTPS channels.

The client browser connects to the UI server, which forwards requests to the application server. The application server, in turn, initiates outbound requests to Azure DevOps and Gemini APIs as needed. This layered approach ensures that sensitive operations remain isolated within the cloud perimeter and that external dependencies are accessed through controlled, authenticated channels.

4.2.7 Connection Graph

Figure 4.3 shows the full connection graph: which UI services invoke which backend modules, which modules access which Azure DevOps resources, and where the optional LLM path branches off.

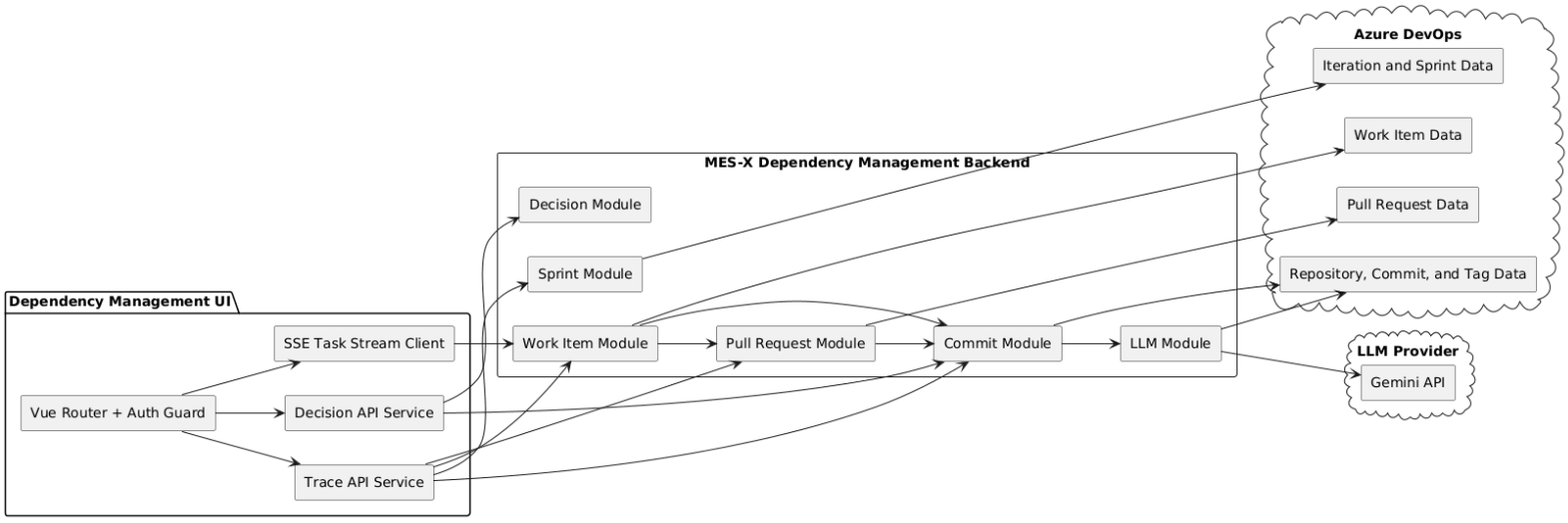

Figure 4.3: Global connection graph of the Trace UI and backend modules

4.3 Design Principles

Five principles guided the design of this system. Each one responds to a practical challenge identified during analysis.

4.3.1 Modularity

The system is divided into independent modules, each responsible for a well-defined task. Changes to one module, such as updating how commits are resolved, do not require touching others like sprint retrieval or decision aggregation. This keeps the codebase readable and reduces unintended side effects when the system evolves.

4.3.2 Separation of Concerns

Modules communicate only through well-defined data transfer objects (DTOs). No module reaches into another's internal logic. This boundary is what makes the system predictable: each module can be understood and tested in isolation, and the interfaces between them are explicit.

4.3.3 Data Normalization

Azure DevOps responses are not always consistent across projects, teams, or API versions. Before processing, data is converted into a stable internal format. This normalization layer acts as a buffer between raw API output and the rest of the system, so business logic does not need to account for external inconsistencies.

4.3.4 Batch Processing

Rather than fetching items one by one, the system groups requests wherever possible. This reduces API traffic, speeds up analysis, and helps avoid Azure DevOps rate limits. For large projects with hundreds of work items and dozens of repositories, this is the difference between a responsive tool and an unusable one.

4.3.5 Assisted Decision Support

A clear line separates deterministic traceability logic from optional AI assistance. Release-readiness decisions are always computed from Azure DevOps data using fixed rules and never depend on the LLM. The LLM path exists only to provide additional review insight on selected commits, as a complement to core logic.

4.4 Core Components

The backend is organized into six components that form a linear processing pipeline:

Sprint Selection → Work Items → Pull Requests → Commits → Decisions (+ Optional LLM Review)

Each stage adds a layer of context to the data before passing it to the next.

4.4.1 Sprint Exploration Component

This component provides a planning-oriented entry point into the traceability pipeline. It loads available sprints for a project and retrieves the work items associated with the selected sprint. The resulting work item IDs are then handed off to the same pipeline used for manual selection, so the rest of the flow is identical regardless of how analysis started.

4.4.2 Work Item Management Component

This is where the dependency graph takes shape. Starting from one or more work items, the component explores all relationships (parent/child hierarchies, related items, and pull request links) and expands them recursively until the full dependency tree is collected.

To keep this process efficient, it relies on graph traversal with batching and caching, so the same item is never fetched twice and API calls are grouped wherever possible. The output is a complete list of related work items, along with associated pull requests.

4.4.3 Pull Request Resolution Component

Pull requests are the bridge between the planning world (work items, sprints) and the code world (commits, branches). This component fetches details for each pull request identified in the previous step and maps them to their repositories.

Without this step, it would be impossible to know which code changes correspond to which features.

4.4.4 Commit Aggregation Component

This component retrieves the actual code changes. It collects commits from pull requests and direct links, removes duplicates across repositories, and augments each commit with branch and tag information so each change can be positioned in the release timeline.

It also exposes the review entry point used by the decision view: when a user wants LLM feedback on a specific commit, this component handles the handoff to the LLM module.

4.4.5 Decision Aggregation Component

Rather than fetching new data, this component analyzes everything collected in previous stages and produces a structured release-readiness assessment. It answers questions such as:

- Are all work items in the dependency graph completed?
- Are the relevant commits included in the current release branch or tag?
- Which repositories are in a ready state, and which ones have outstanding issues?

The output is a clear, per-repository decision view that teams can use directly during release evaluation.

4.4.6 LLM-Assisted Review Component

This component is entirely optional and only runs when explicitly requested. Given a specific commit, it builds a review prompt using commit metadata, changed files, and any configured repository-level guidance. It then calls the external model and returns the review payload to the UI.

This component does not influence deterministic decisions produced by the previous stage. It exists as a separate, on-demand tool for teams that want an additional layer of code-level analysis.

4.5 End-to-End Data Flow

From the moment a user starts an analysis to the moment results appear on screen, the system follows a consistent sequence:

1. The user initiates an analysis from the Trace UI, either by selecting work items manually or by choosing a sprint.
2. If sprint mode is used, the system fetches the sprint's work items and forwards their IDs into the pipeline.
3. The Work Item component retrieves each item and expands the full dependency graph.
4. The Pull Request component resolves the pull requests linked to those work items.

5. The Commit component collects all commits, deduplicates them, and adds branch and tag context.
6. The Decision component aggregates everything into a release-readiness view.
7. Optionally, the user can trigger an LLM review for specific commits from within the decision view.
8. All results are displayed in the Trace UI's trace and decision views.

At each step, the data becomes richer and more meaningful. Raw work item IDs become a dependency graph; commits are placed in a release timeline; individual statuses become an aggregated readiness assessment.

4.6 User Interface and Interaction

The Trace UI is designed around the four usage modes described earlier: single item analysis, batch analysis, sprint exploration, and batch-related analysis. Each mode is available from the same interface and feeds the same backend pipeline.

One deliberate design choice was to use streaming for real-time feedback. Instead of showing a single loading state while the backend completes the full analysis, the UI reports progress as each stage finishes. For large analyses, this noticeably improves the user experience.

From the decision view, users can also trigger an LLM commit review directly, without leaving the interface. This keeps the workflow in one place instead of requiring users to copy commit hashes into another tool.

4.7 Observability and Logging

The backend uses a custom `AppLogger` class for structured logging throughout the system. This logger extends the shared logging service used in the project and is initialized through the logger configuration module so that the minimum log level can be loaded from the environment or from the application configuration store.

This makes it easier to understand what happened during an analysis and to diagnose problems when they occur. Four log levels are used:

- `log` for normal operations,

- `warn` for minor issues that do not break the analysis but are worth noting,
- `error` for failures that affect results,
- and `debug` for detailed internal state, useful during development.

Taken together, the components and design decisions described in this chapter form a system that transforms scattered Azure DevOps data into a structured, end-to-end traceability view. The combination of deterministic dependency analysis, sprint-oriented entry points, and optional AI-assisted review gives teams the visibility they need to make informed release decisions, without having to piece the picture together manually each time.

Chapter 5

Implementation

5.1 Introduction

This chapter covers the technical implementation of the MES-X dependency traceability system. It describes how the proposed architecture was turned into a working full-stack application that brings together work items, pull requests, commits, branches, tags, sprint planning data, and AI-assisted review into a single coherent workflow.

The chapter presents the technology stack, the delivery phases, the backend module design, the frontend integration, the deployment setup, and the quality practices used to validate the final product.

5.2 Technology Stack

MES-X is built on a full-stack architecture designed to handle large-scale dependency graph reconstruction and traceability analysis.

5.2.1 Frontend Layer

The frontend is the main interface operators use to run and explore traceability analyses. It is built for responsiveness and clarity when rendering complex dependency graphs.

- **Framework:** Vue 3 with TypeScript
- **UI components:** Quasar
- **Build tool:** Vite

Figure 5.1: Vue 3 – frontend framework

5.2.2 Backend Layer

The backend handles all traceability logic and exposes structured APIs for both single-root and batch analysis.

- **Framework:** NestJS (TypeScript)
- **API styles:** REST for queries, SSE for streaming

The backend is organized into domain-driven modules, each responsible for processing and enriching a specific part of the traceability data.

Figure 5.2: NestJS – backend framework

5.2.3 Domain Service Layer

This layer holds the core business logic of the system:

- work item graph traversal and relationship resolution;

- pull request linkage and analysis;
- commit extraction and repository context resolution;
- sprint-to-work-item retrieval for seeding batch traces;
- LLM-assisted commit review with repository-specific guidance;
- tag and branch contextual enrichment;
- deduplication and cycle prevention across traversal paths.

5.2.4 Cloud Infrastructure and DevOps

- **Source control:** Azure DevOps Git repositories
- **CI/CD:** automated build, test, and deployment pipelines
- **Containerization:** Docker for reproducible environment execution
- **External APIs:** Azure DevOps Work Item and Git APIs

Figure 5.3: Azure DevOps – source control and CI/CD platform

5.3 Methodology Execution Through Delivery Phases

5.3.1 Methodology Selection

Because this project combines evolving requirements, multi-service integration, and short feedback loops with operational teams, selecting an appropriate working methodology was a key project decision.

Three candidate methodologies were evaluated against the project context:

Table 5.1: Comparative analysis of candidate project methodologies

Methodology	**Main Advantages**	**Main Limitations**	**Fit for This Project**
Waterfall	Strong structure, clear documentation milestones, simple governance	Low adaptability to changing requirements, late feedback on integration issues	Limited fit
Kanban	Continuous flow, high flexibility, low process overhead	Weaker timeboxed milestones, less explicit iteration framing for internship checkpoints	Partial fit
Agile Scrum	Timeboxed iterations, frequent feedback loops, clear review cadence, risk reduction through incremental delivery	Requires disciplined backlog grooming and sprint planning	High fit

Waterfall offers strong phase-by-phase structure and clear documentation gates, which is useful for stable projects with fixed scope. However, it is less suitable here because dependency-traceability rules and integration details had to be refined progressively based on intermediate validation.

Kanban provides continuous flow, operational flexibility, and low process overhead. Its main limitation for this project is weaker iteration framing for milestone-based evaluation, which is important in an internship context where progress must be demonstrated at regular checkpoints.

Agile Scrum combines iterative delivery, explicit planning horizons, and regular review cycles. It supports incremental integration of backend, frontend, and Azure DevOps connectors while allowing early correction of design and implementation choices.

5.3.2 Selection Rationale

Based on this comparison, **Agile Scrum** was selected as the primary methodology. This choice was motivated by technical and organizational criteria:

- the need to deliver working increments quickly (work-item tracing, pull-request resolution, commit/tag contextualization, sprint analysis);

- the need for frequent validation with end users and release engineers;
- the need to reduce integration risk through short implementation and test cycles;
- the need for visible milestone tracking across the internship timeline.

The enterprise environment already supported agile tooling through Azure DevOps Boards, which made Scrum execution practical. Nevertheless, the methodology was adopted for its demonstrated fit to project constraints and goals, not as an unquestioned default.

5.3.3 Application to Delivery Phases

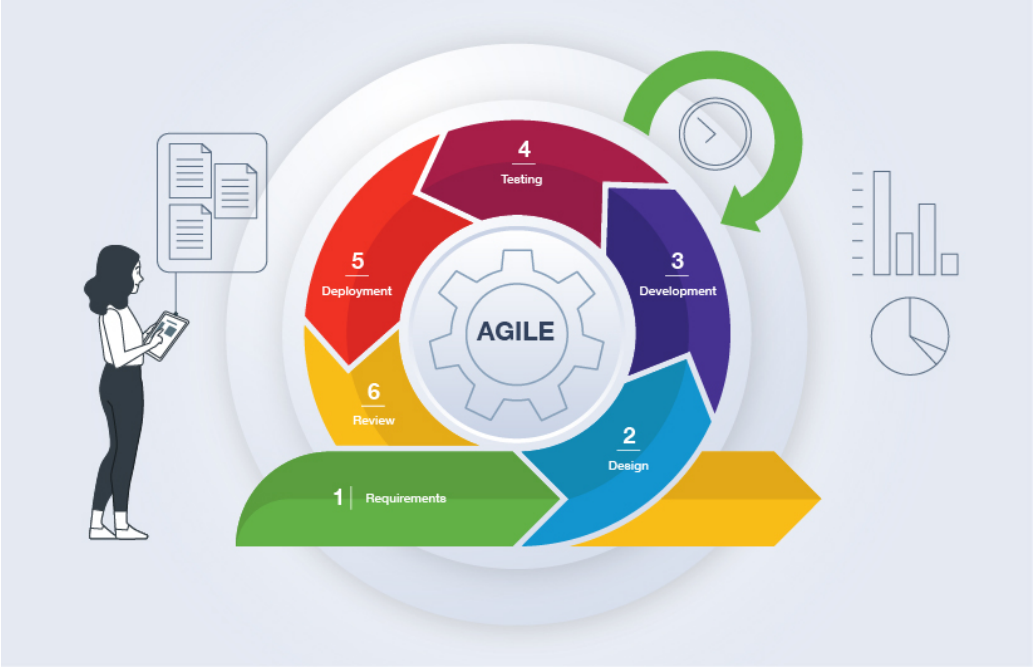

Figure 5.4: Agile iterative cycle applied to delivery phase planning

Each module went through the full software lifecycle: design, implementation, testing, integration, and deployment. This kept integration risks low by continuously aligning the backend, frontend, LLM features, and DevOps pipelines.

Phase 1 – Setup and Infrastructure

- Developer onboarding and environment preparation.
- Local infrastructure setup and database connections.
- CI/CD pipeline initialization.

- Frontend project structure and build pipeline creation.

Outcome: fully operational development environment, automated build pipeline, and initial frontend architecture.

Phase 2 – Work Items Module

Backend: work item data model, service layer, REST endpoints, validation and schema enforcement.

Frontend: listing and filtering interface, creation and editing forms, detailed view with status tracking.

Delivery: backend unit tests, end-to-end UI tests, staging and production deployment.

Phase 3 – Pull Requests Module

Backend: pull request data model, storage and filtering logic, API endpoints, statistical enrichment.

Frontend: PR dashboard, diff viewer, review and approval workflow, status indicators and activity timeline.

Delivery: API and data model documentation, unit and integration tests, staging validation and production rollout.

Phase 4 – Commits and Graph Traceability Engine

This phase introduced the core traceability engine, enabling deep graph exploration across commits, branches, and repositories.

Backend: commit data model, BFS/DFS graph traversal, cycle detection, performance tuning for large repositories.

Frontend: commit history view, interactive dependency graph visualization, commit details and file-level diff inspection.

Validation: unit tests for traversal and API logic, end-to-end graph tests, performance validation on large datasets.

Phase 5 – Sprint Module

Backend: sprint listing backed by Azure DevOps iterations, work item retrieval through iteration relations, DTO contracts, logging and error handling.

Frontend: sprint exploration screen, lazy loading of work items per expanded sprint, cross-sprint selection before batch analysis.

Validation: controller and service unit tests, manual flow validation, integration with the batch traceability workflow.

Phase 6 – LLM-Assisted Review

Backend: dedicated LLM module integrated into the Commit module, prompt construction from commit metadata and changed files, injection of repository-specific guidance, result caching and retry handling.

Frontend: LLM review action exposed from the decision dashboard, review dialog for displaying findings, coupling between decision results and commit review requests.

Safeguards: human validation remained mandatory; prompt scope was limited to relevant files; LLM failures were isolated from the core traceability workflow.

Phase 7 – System Release

- Integration of all modules into a unified system.
- Automated CI/CD validation stages.
- Production deployment pipeline configuration.
- Post-deployment monitoring and validation.

Outcome: fully integrated MES-X traceability platform, stable production pipeline, and validated end-to-end traceability workflow.

5.4 Validation and Deployment

Testing was carried out at three levels. Backend unit tests covered controller and service behavior for the Work Item, Commit, and Sprint modules, including tag-selection logic. A backend end-to-end test suite validated core HTTP integration under the `test/` directory. Frontend validation combined linting, static analysis, and manual walkthrough of the main traceability screens and the decision dashboard.

Once validation passed, the system was packaged as a Docker image and deployed through the Azure DevOps pipeline. Figure 5.5 shows the pipeline execution with the Build and Dev stages completing successfully.

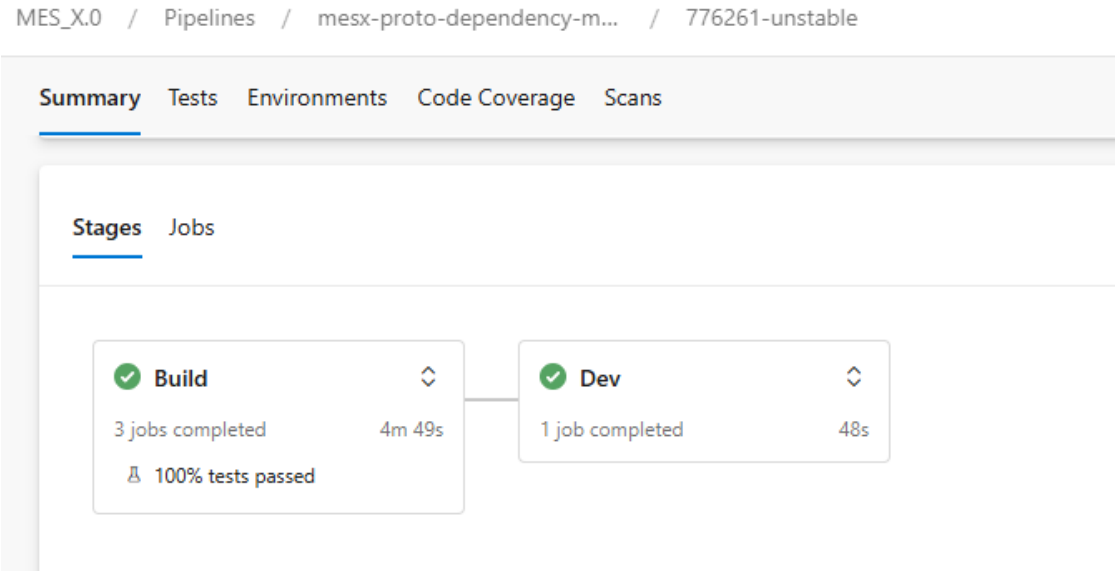

Figure 5.5: Azure DevOps pipeline – Build and Dev stages after deployment

Post-deployment checks confirmed endpoint availability and correct frontend-backend integration across the main traceability workflows.

5.5 Backend Design

5.5.1 Module Architecture

The backend is organized into six domain modules:

- **Work Item:** single-root and batch traversal endpoints, SSE task streaming.
- **Pull Request:** linked work item retrieval by PR, PR summary enrichment.
- **Commit:** commit lookup by work item and PR, branch and tag context retrieval, batch tag resolution, commit review orchestration.
- **Sprint:** sprint listing by project, sprint work item retrieval for seeding batch analysis.
- **Decision:** aggregation service that synthesizes traceability outputs for decision support.

- **LLM:** prompt generation, guidance collection from repository instructions, external model invocation, response caching.

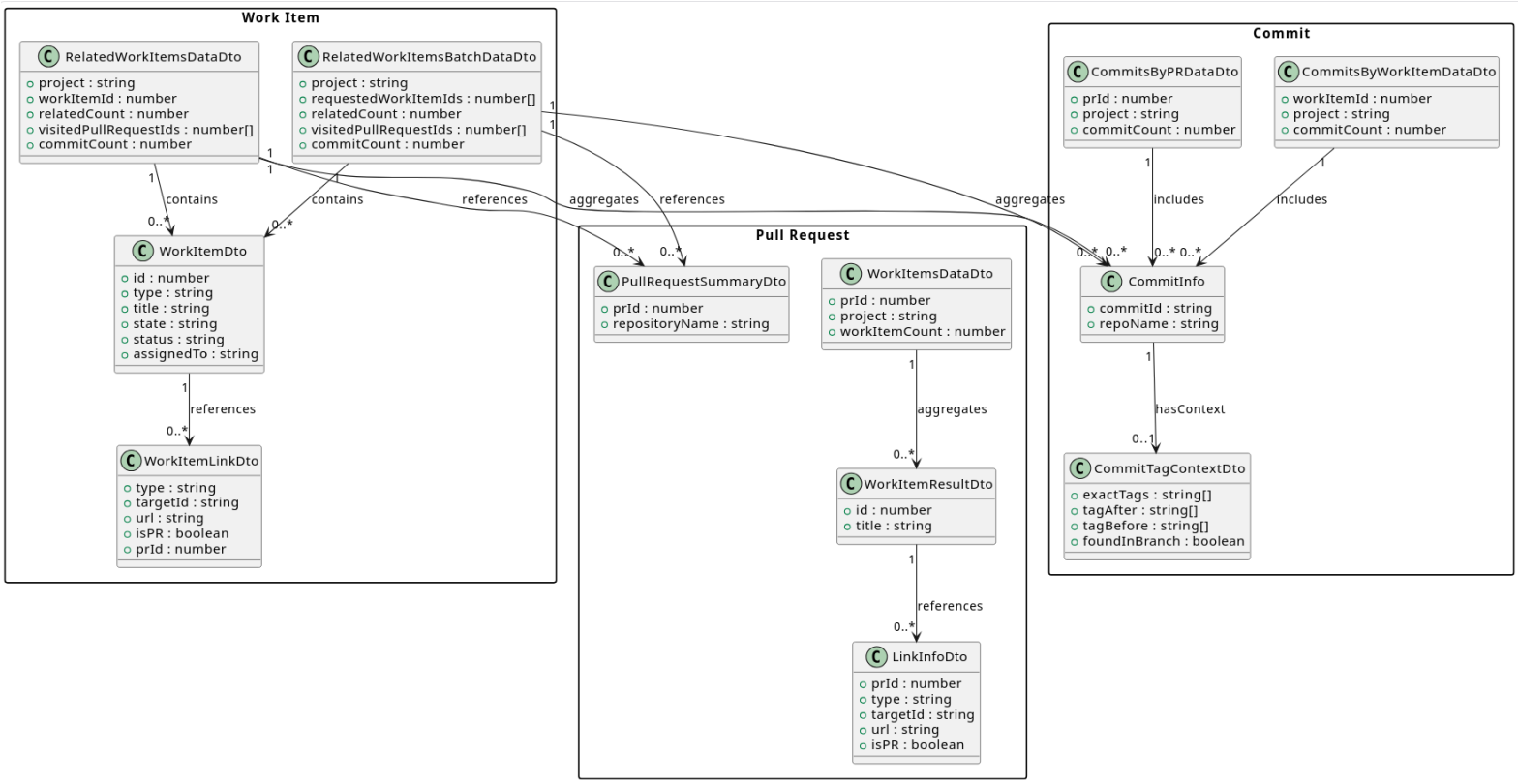

Figure 5.6: Class diagram – Work Item, Pull Request, and Commit modules

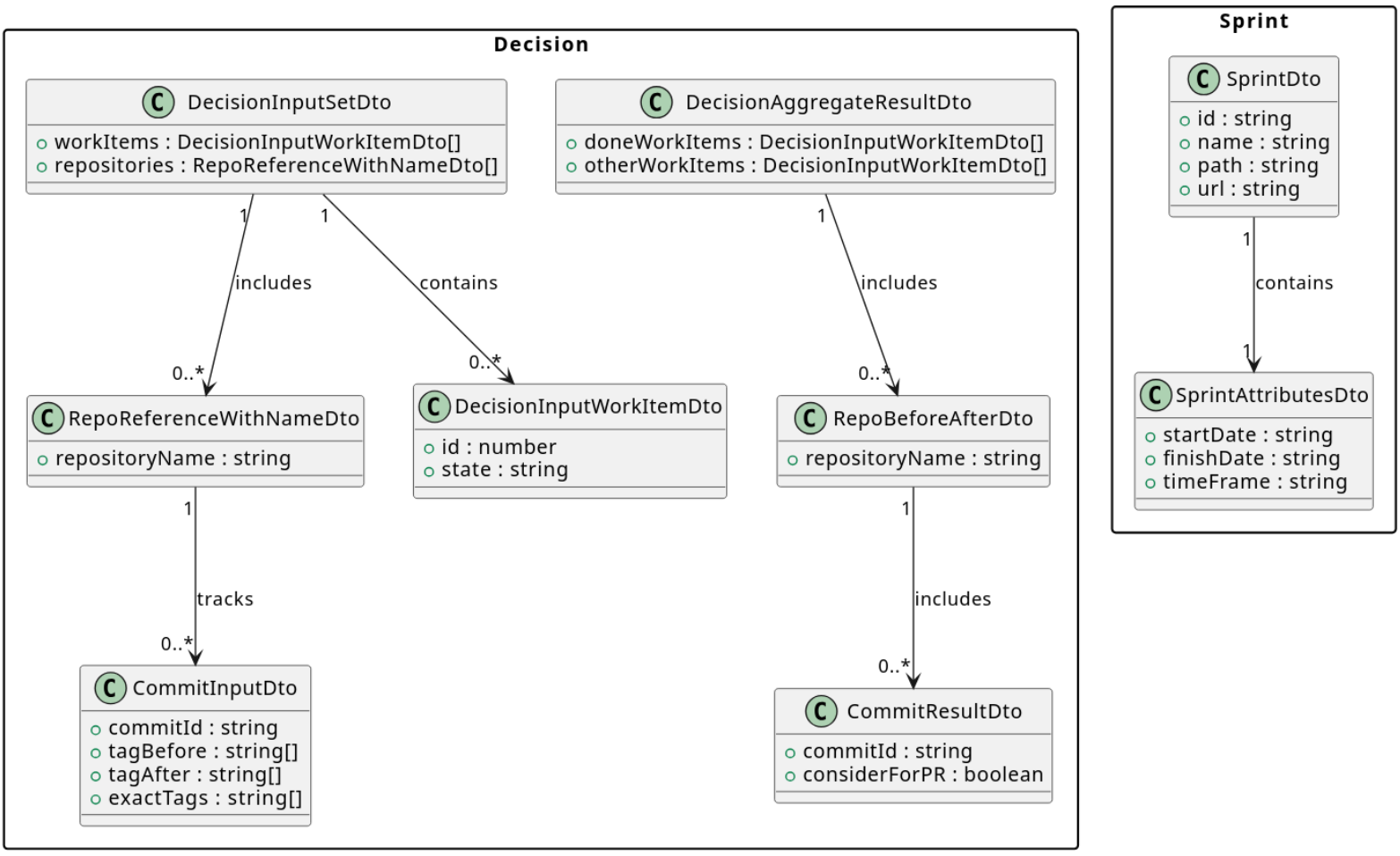

Figure 5.7: Class diagram – Sprint and Decision modules

5.5.2 API Design

The backend exposes RESTful endpoints supporting single-root and batch traceability with a consistent response model.

`GET /workitem/:id/related` Returns the traceability graph for a single root work item.

`POST /workitem/related` Returns a deduplicated, aggregated graph for multiple root work items.

`GET /workitem/tasks/stream` Streams processing progress using Server-Sent Events (SSE).

`GET /pullrequests/:prId/workitems` Returns work items linked to a given pull request.

`GET /sprint?project=...` Lists the iterations available for a project.

`GET /sprint/:sprintId/work-items?project=...` Returns work items for a sprint; supports lazy loading in the UI.

`POST /commits/tags/batch` Enriches commit references with repository tag metadata.

`POST /commits/hf/review` Generates an AI-assisted review for a selected commit using commit and repository context.

DTO contracts across all endpoints prioritize stable response structures, best-effort metadata enrichment, and a clear separation between traceability payloads and AI-generated review content.

5.5.3 Processing Flows

- **Single-root:** resolve the root work item, traverse related nodes, enrich pull requests and commits, return a normalized graph.
- **Batch:** multi-root traversal with deduplication and consolidated counters.
- **Sprint:** load project sprints, lazily fetch sprint work items, forward selected items to the batch flow.
- **Commit tag:** resolve tag context in batch, grouped by repository, using a latest-tag strategy.

- **LLM review:** collect commit metadata and relevant file snapshots, augment the prompt with repository guidance, invoke the model, return the generated review to the decision dashboard.

5.6 Frontend Implementation

5.6.1 Route and View Structure

The UI is organized around route-driven views, each mapping to a key analysis scenario:

- **RelatedWorkItemTrace:** single-root analysis – explore one work item and its related artifacts.
- **BatchWorkItemTrace:** batch selection screen with SSE-driven progressive loading.
- **SprintWorkItemsView:** planning-oriented screen for loading sprints, lazily fetching work items, and preparing cross-sprint selections.
- **BatchRelatedWorkItemTrace:** consolidated multi-root analysis result with commit and repository context.
- **DecisionDashboard:** promotion decision screen with commit-level LLM review actions.

5.6.2 Backend Integration

- Single analysis calls `/workitem/:id/related`.
- Batch task loading consumes `/workitem/tasks/stream` via SSE.
- Sprint screens call `/sprint` and `/sprint/:sprintId/work-items` with lazy retrieval per sprint.
- Batch result enrichment uses `/commits/tags/batch` grouped by repository.
- LLM review is triggered from the decision dashboard via `/commits/hf/review`.
- Authentication failures redirect the user when the backend returns an unauthorized response.

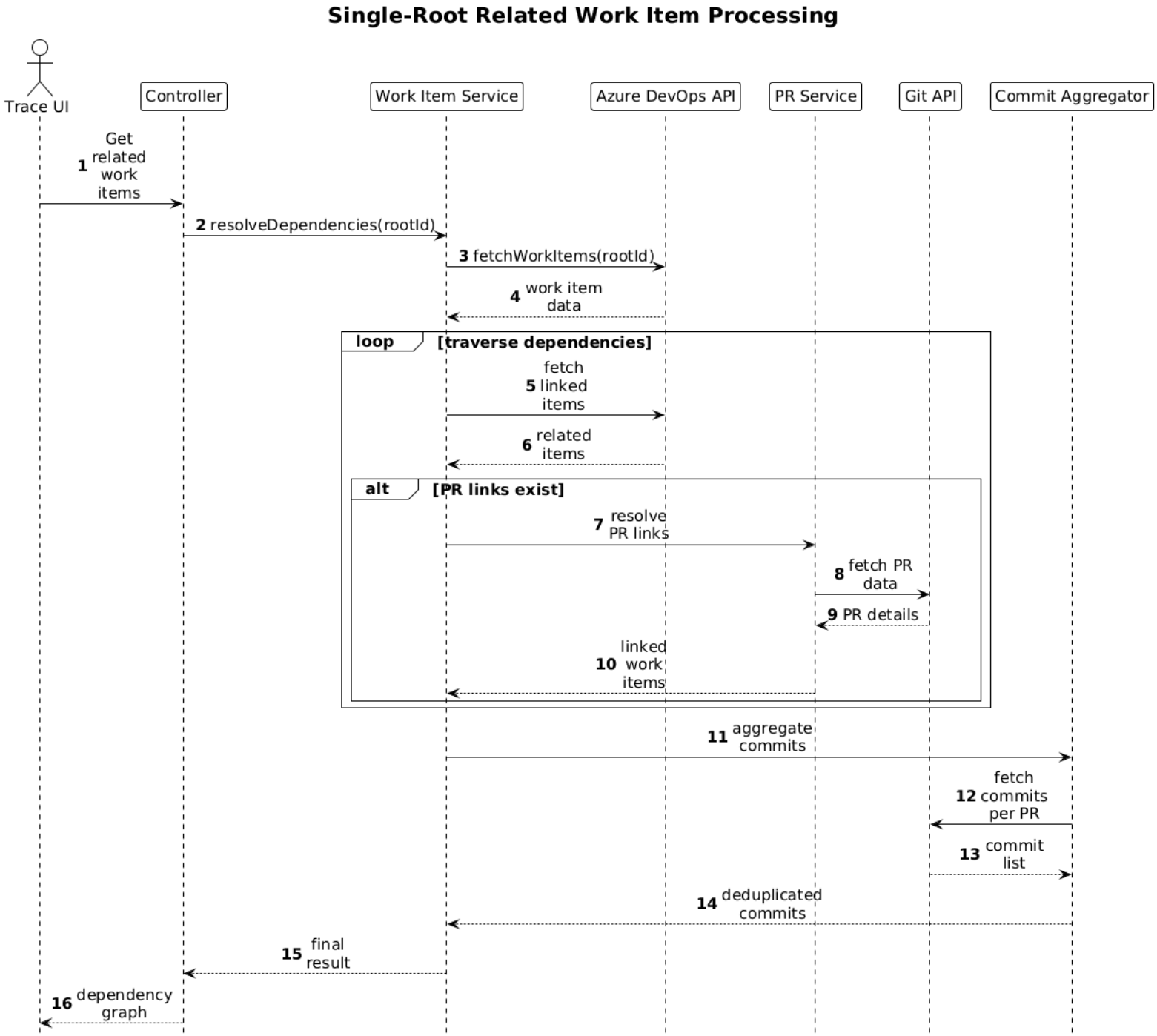

Figure 5.8: Sequence diagram – single-root work item processing

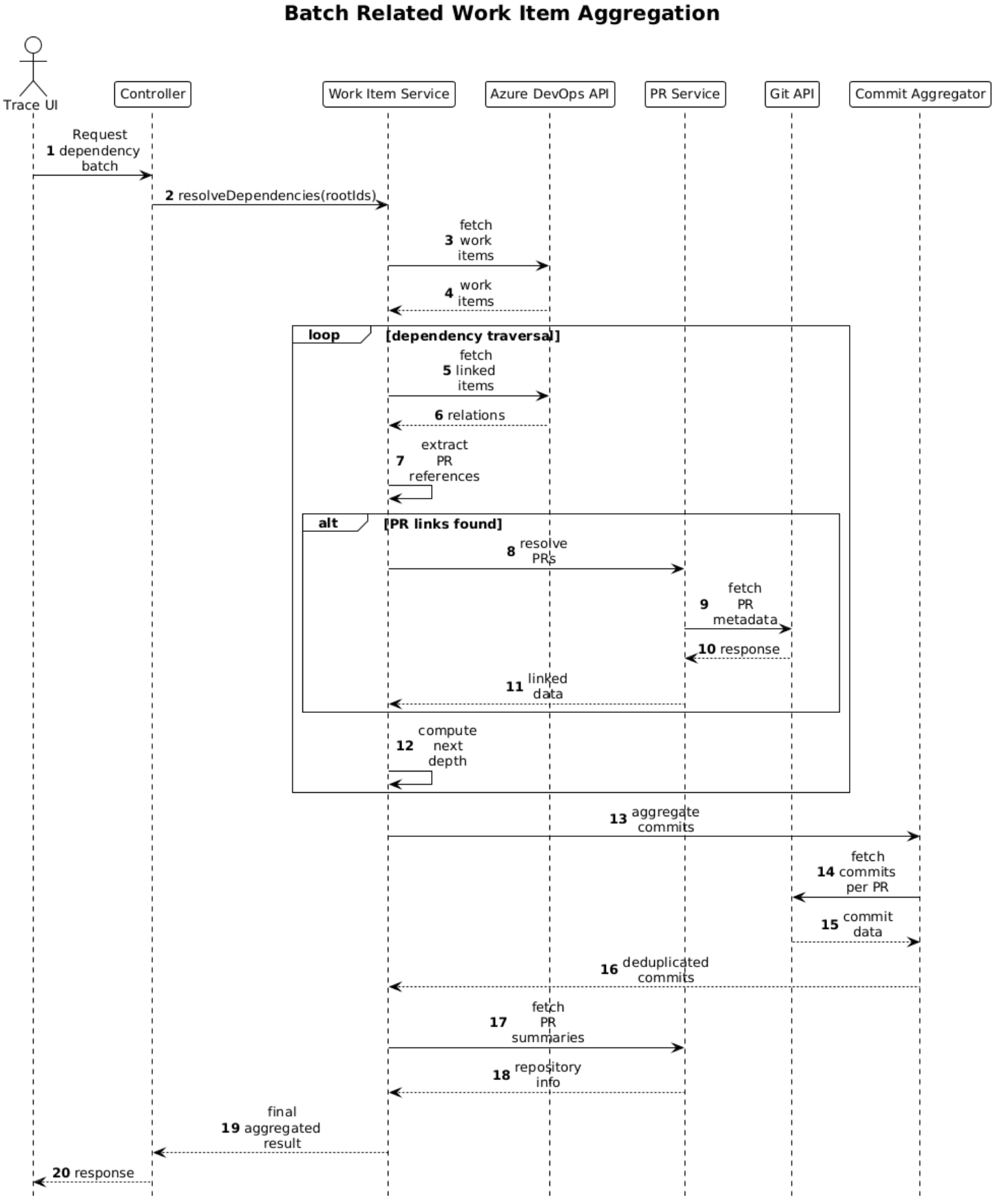

Figure 5.9: Sequence diagram – batch work item aggregation

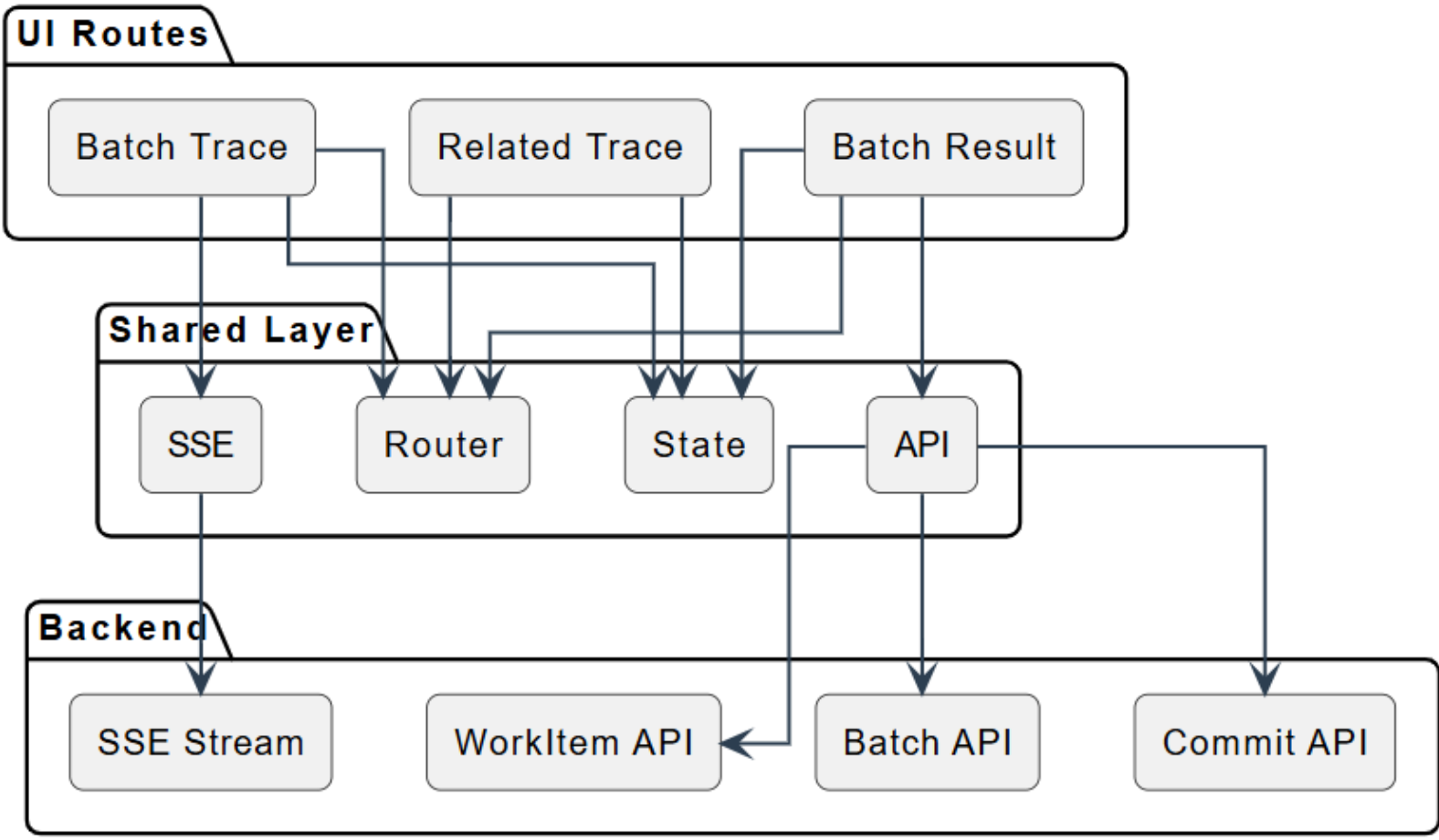

Figure 5.10: UI route and component architecture

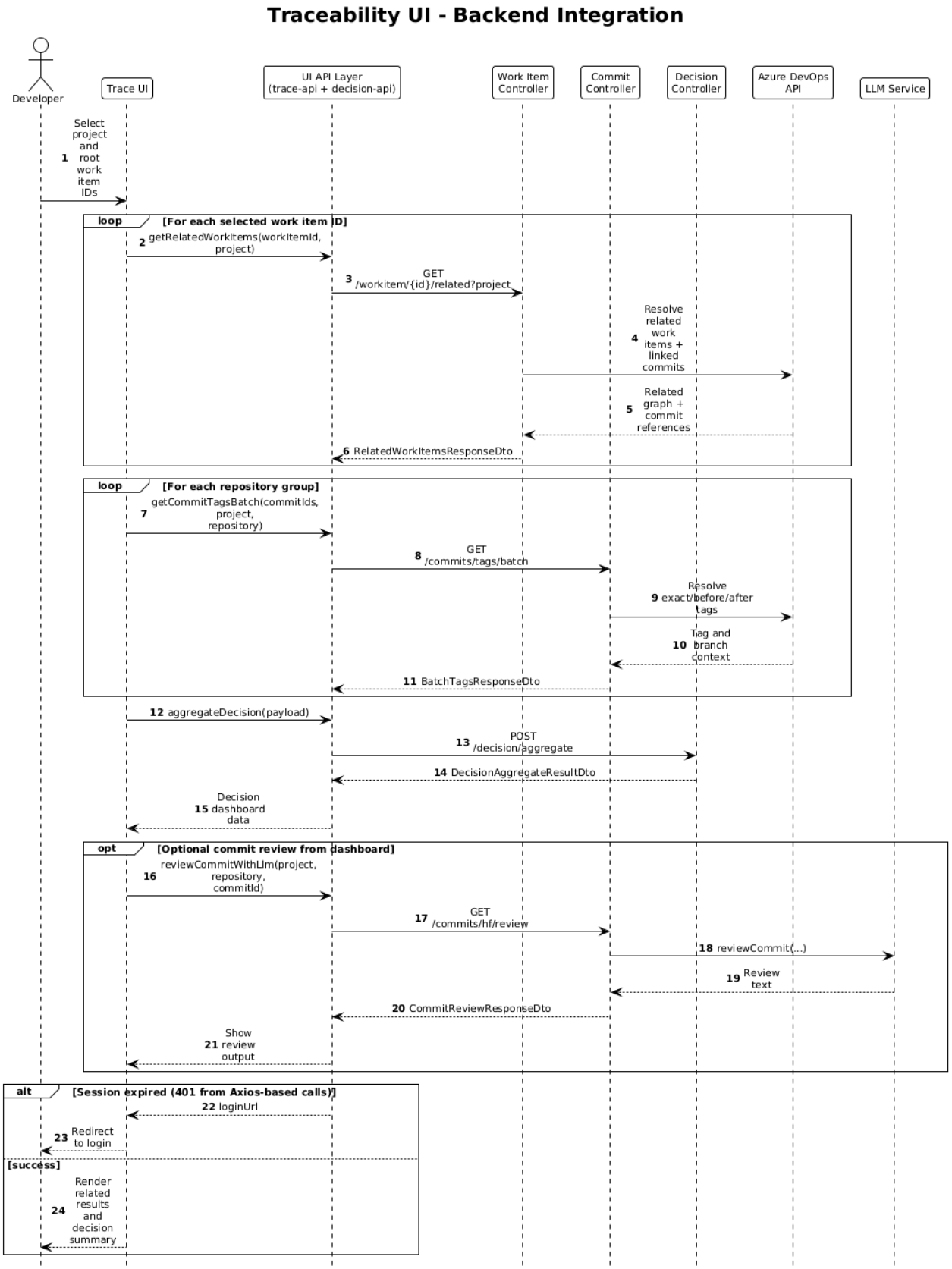

Figure 5.11: Sequence diagram – UI and backend integration

5.6.3 UI Walkthrough

Batch Work Items

The batch screen lets operators select one, several, or all candidate work items and launch a traceability run. Progress is shown progressively via SSE. Quick filters help narrow candidates before the batch is forwarded to the aggregation flow.

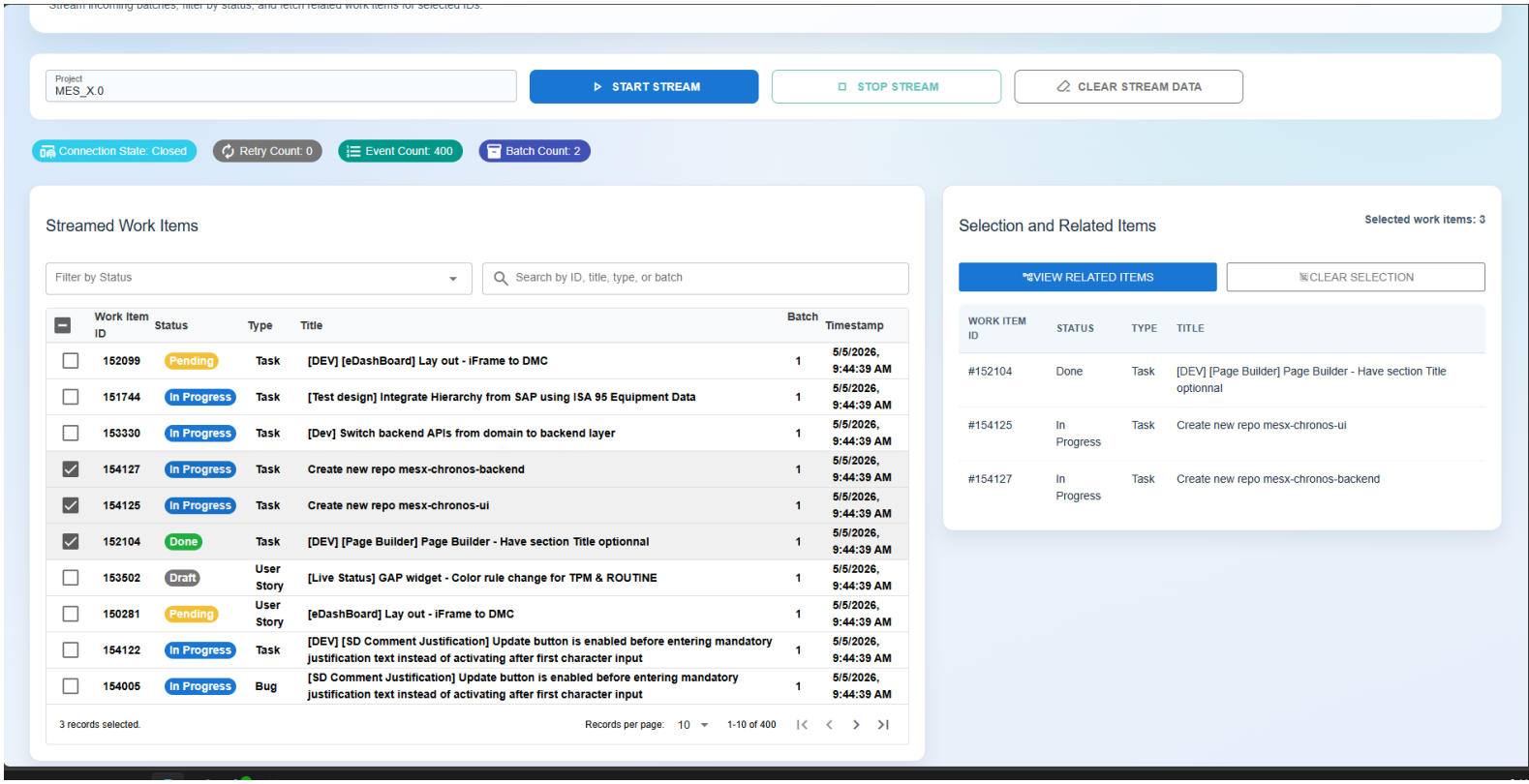

Figure 5.12: Batch work items – selection and SSE-driven progress

Related Work Item

This view shows a single root work item and its connected graph: pull requests, commits, branches, and tags. Nodes can be expanded and inspected down to commit or PR level.

The discovered commits view lists commits found during traversal alongside branch, tag, and work item context. It is useful for verifying commit-to-work-item linkage before any promotion decision is made.

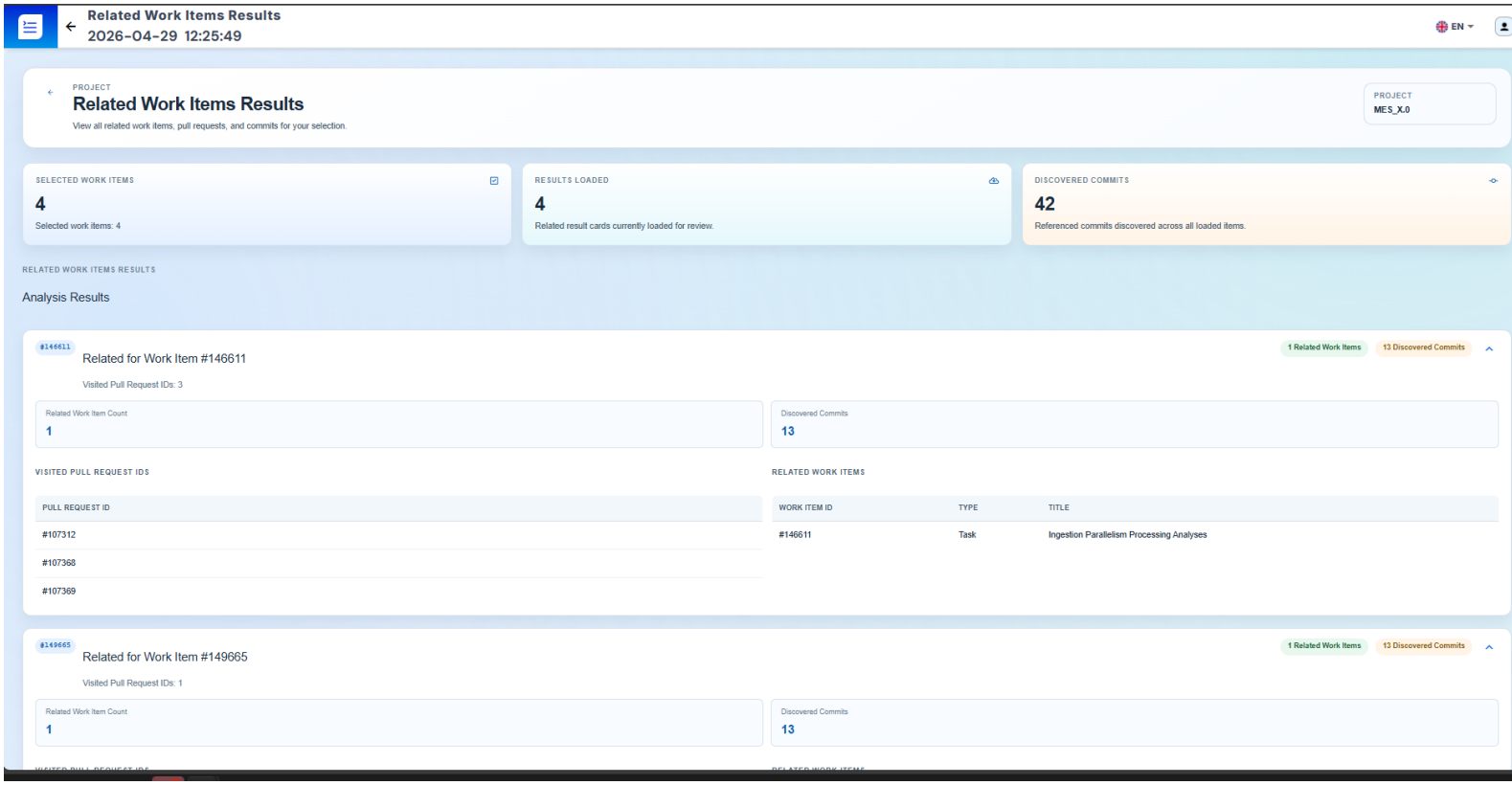

Figure 5.13: Related work item – single-root traceability graph

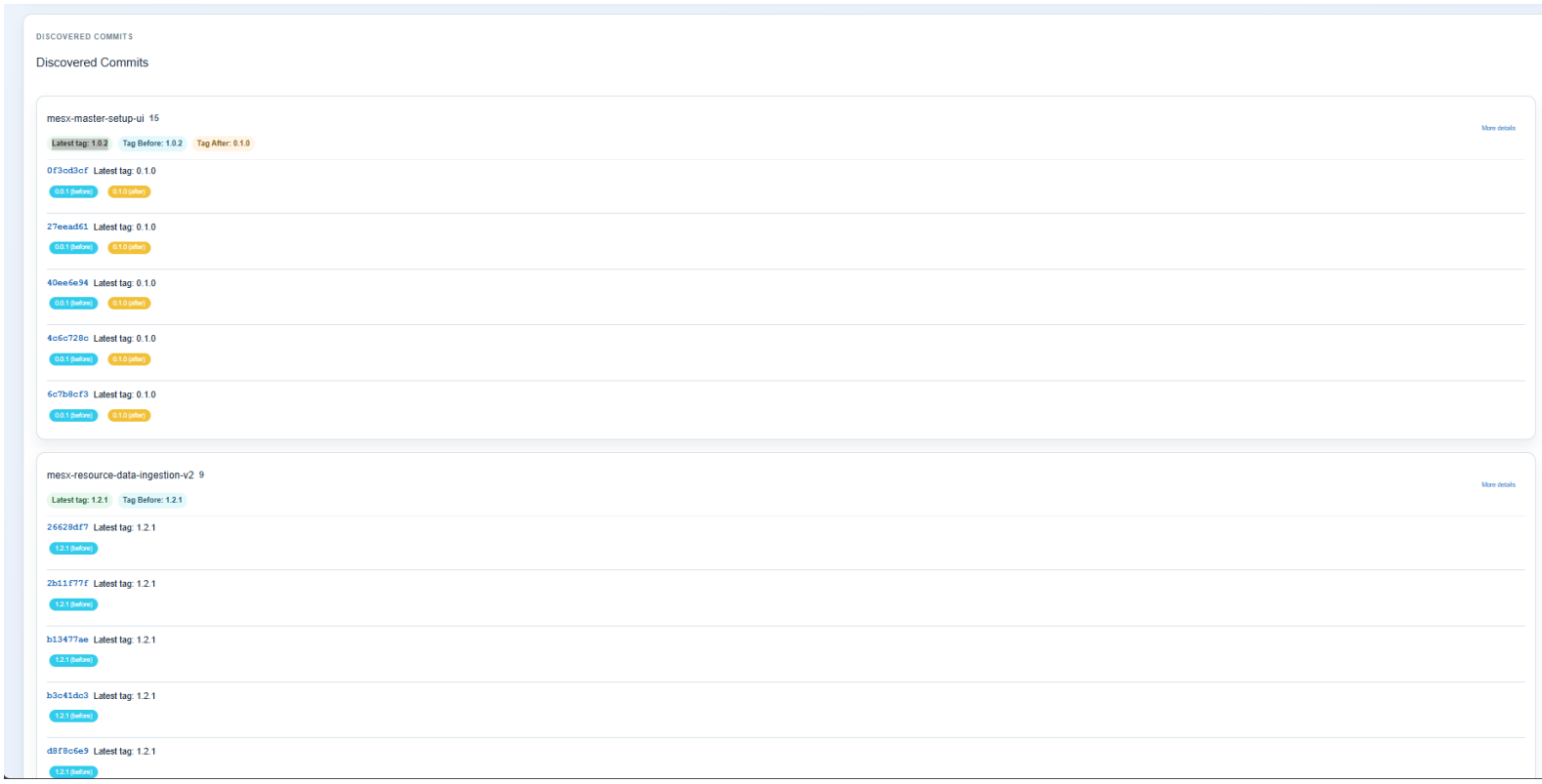

Figure 5.14: Discovered commits – commits found during graph traversal

Sprint Work Items

Rather than entering identifiers manually, operators load the sprint list for a project and expand individual sprint cards to lazily fetch their work items. Selections persist across sprints and are forwarded directly to the batch analysis flow.

Decision Dashboard

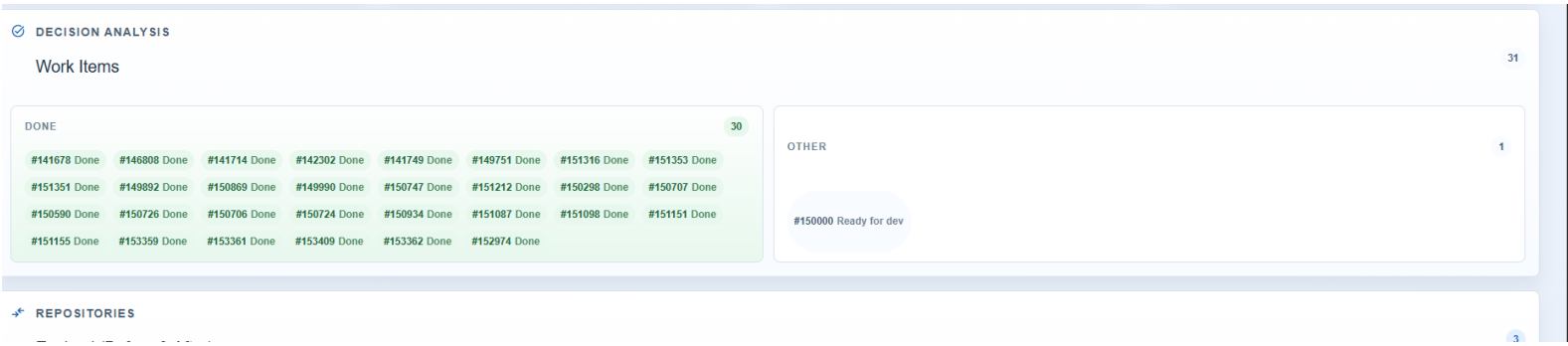

Figure 5.15: Decision – work items grouped by state

The first decision view groups work items by state (`Done` vs. `Other`) so operators can see at a glance which items are ready for promotion.

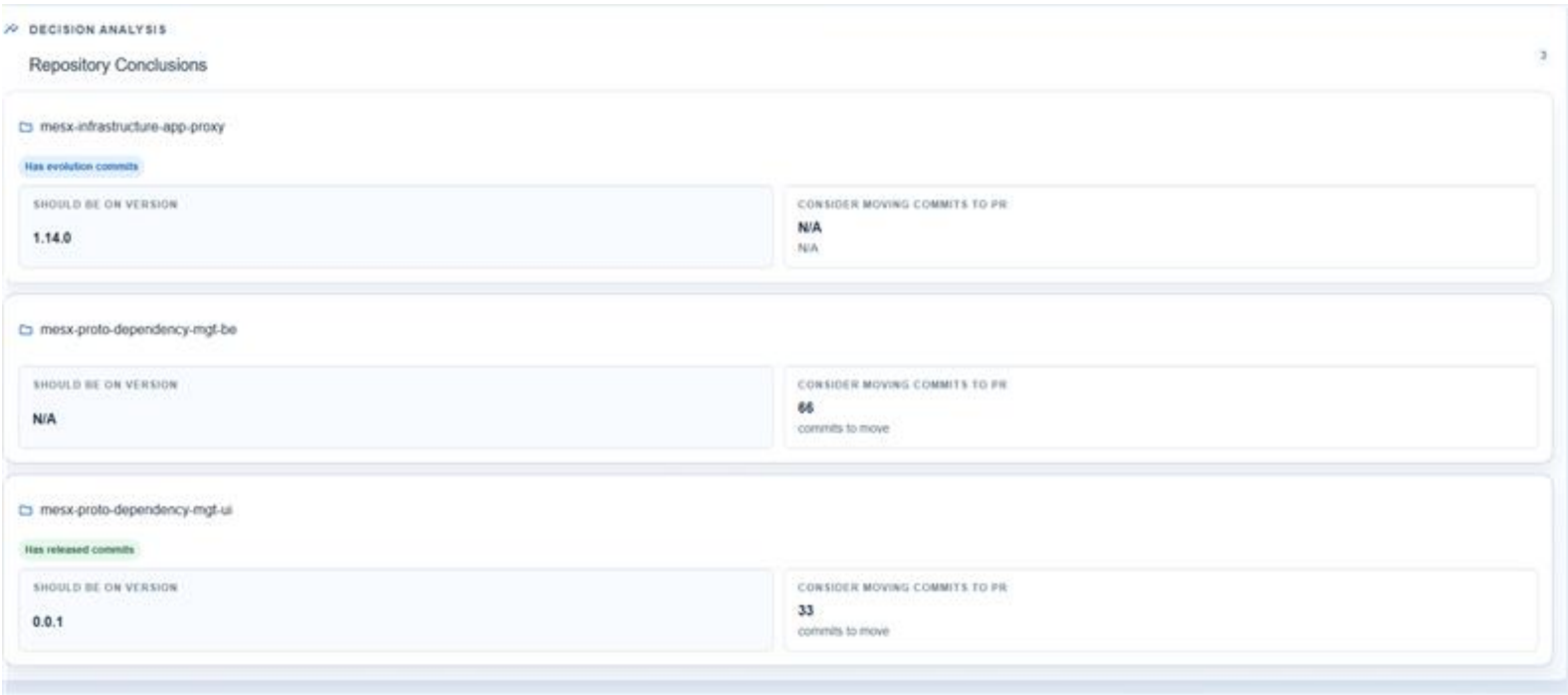

Figure 5.16: Decision – repository view with expected tags for promotion

The repository and tags view shows, per repository, the commits and expected tags (before, after, exact) identified by the system. From here, operators can review tag

context, trigger a promotion workflow, or request an LLM-generated commit review before making a final decision.

5.7 Deployment

5.7.1 Container Model

The backend is packaged as an isolated Docker container with clearly defined integration boundaries.

- REST and SSE APIs are exposed on the configured runtime port.
- Runtime configuration and secrets are injected through environment variables and externalized config files.
- All communication with Azure DevOps is routed through the backend over HTTPS.

5.7.2 Deployment Objectives

- Reproducible execution locally and in CI.
- Independent versioning and rollout of backend artifacts.
- Reduced deployment risk through clear component ownership.

5.8 Quality and Security

5.8.1 Testing

Validation was carried out at multiple levels:

- **Backend unit tests (Jest):** controller and service behavior for the Work Item, Commit, and Sprint modules, including tag-selection utility functions.
- **Backend end-to-end test:** core HTTP integration behavior validated under the `test/` suite.
- **Frontend validation:** linting, static checks, and manual scenario testing of the main traceability screens and decision dashboard.

- **Cross-layer checks:** streaming behavior, repository grouping, and decision outputs verified end-to-end from backend to UI.

5.8.2 Security Controls

- PAT-based authentication for all Azure DevOps API access.
- Backend-only routing of external API calls – no direct frontend access to Azure DevOps.
- Secrets and credentials managed through environment configuration and secret store patterns.
- Read-only enforcement: the system issues no write operations to Azure DevOps artifacts.
- Structured logging to support incident analysis and diagnostics.

5.9 AI-Assisted Development

AI tools were used throughout the project to support both backend and frontend work – not to replace engineering decisions, but to speed up repetitive tasks, improve review coverage, and enforce project conventions.

Both repositories keep automation artifacts versioned with the source code. The backend `.github/agents/` directory defines dedicated agent roles: architect, implementer, reviewer, and QA. The frontend `.github/` folder captures UI guidelines, review rules, and reusable workflows for Vue, Quasar, styling, and quality checks.

In practice, these agents assisted with code drafting, refactoring, test scaffolding, documentation, and architecture reviews. On the backend they kept DTO usage, controller-service separation, and observability patterns consistent. On the frontend they reinforced guardrails around Vue Composition API usage, token-based styling, and component quality.

The system also ships an operational LLM capability in the backend. The Commit module delegates review requests to the LLM module, which builds prompts from Azure DevOps commit metadata, selects relevant changed files, injects repository guidance, and calls an external model to produce a structured review. This review surfaces in the decision dashboard to complement release analysis with code-level observations.

All AI-generated suggestions went through human review. Proposed changes were treated as drafts and validated through linting, SonarQube policies, tests, and developer inspection before being merged.

5.10 Conclusion

This chapter described how the MES-X traceability system was built from infrastructure setup through to production deployment. The implementation followed a phased delivery approach, with each module integrated and validated incrementally. The result is a stable, maintainable platform that covers the full traceability chain from sprint planning to commit-level review.

General Conclusion

This internship addressed a real problem: release preparation at Forvia Informatique Tunisie was slow and spread across multiple Azure DevOps screens with no unified view of how artifacts related to one another. The work covered the full development cycle, from architecture and implementation to integration, validation, and deployment.

The result is a modular traceability platform. The backend handles recursive graph traversal across work items, pull requests, commits, and tags, with cycle prevention, batch processing, and SSE streaming for progressive feedback. The frontend gives operators a single flow from work item selection to promotion-ready summaries, replacing a process that previously required navigating several disconnected pages. A lightweight LLM review feature was also integrated at commit level, keeping human validation as the final step.

Beyond the delivered features, the project confirmed a few practical lessons. Incremental module delivery kept integration risks low. Strict DTO contracts between backend and frontend prevented most regressions. Logging and structured error handling proved essential once flows became multi-step and harder to trace manually.

The platform is functional and deployed, but also designed to evolve. The traceability graph it builds is machine-readable, which makes it a natural foundation for a future planning assistant able to recommend repository change paths and target versions directly from a work item or user story. That next step would take MES-X from dependency exploration to guided release preparation, which is the logical continuation of what was built here.